Variational Autoencoders with Enhanced Capacity and Conditional GANs for Data Augmentation under Limited Data Regimes

Taha Majlesi
Electrical and Computer Engineering University of Tehran
tahamajlesi@ut.ac.ir

February 13, 2026

Abstract

Deep generative models provide a principled framework for learning complex data distributions and have become central to modern unsupervised and semi-supervised learning. Among the most widely studied approaches are Variational Autoencoders (VAEs) and Generative Adversarial Networks (GANs), each offering complementary strengths and limitations.

VAEs are trained by maximizing a variational lower bound on the data log-likelihood and provide an explicit probabilistic model with tractable inference. However, standard VAEs are known to suffer from limited expressiveness and optimization pathologies such as posterior collapse, where the learned latent variables fail to carry meaningful information. Recent work has proposed modified objectives and hierarchical latent architectures to address these issues by increasing model capacity and encouraging effective latent utilization.

GANs, in contrast, learn to generate samples through an adversarial training process and often produce sharper samples but lack an explicit likelihood. When training data are limited, GANs offer a particularly valuable opportunity: they can be used as data generators to augment small datasets and improve the generalization performance of downstream classifiers. Nevertheless, GAN training is notoriously unstable, motivating the development of techniques such as Wasserstein objectives, gradient penalties, spectral normalization, and attention mechanisms.

This report investigates both paradigms in a unified experimental framework. First, we analyze standard and modified VAEs on Dynamic MNIST, focusing on likelihood estimation, latent structure, and posterior behavior. Second, we design a conditional WGAN-GP with attention mechanisms for FashionMNIST and evaluate its effectiveness as a data augmentation tool under constrained data conditions.

Contents

1 Introduction

1.1 Problem Statement

In this report, we address the problem of template-to-photo alignment, where the goal is to geometrically align a 2D reference template image, denoted as $\mathcal{I}_T$, to a photograph $\mathcal{I}_Q$ of the corresponding physical part. The reference template $\mathcal{I}_T$ could represent any annotated 2D object (e.g., a CAD-derived drawing, a part map, or a blueprint), while $\mathcal{I}_Q$ is the photograph that contains the physical part whose geometric correspondence with the template we need to find.

The central challenge lies in computing an accurate geometric mapping between the two images, using a technique that can robustly handle differences in viewpoint, illumination, and background clutter. The final output is a visual overlay that maps the template onto the photograph, enabling visual inspection and/or facilitating downstream measurement tasks such as part alignment, dimensioning, or quality control.

1.2 Motivation

Template-to-photo alignment is a fundamental task in various industrial and engineering applications. For example, in automated quality control, part inspection, and assembly verification, being able to precisely align digital templates (e.g., CAD files or part blueprints) with real-world photographs enables automated systems to assess whether physical components match their specifications. However, this task is complicated by several real-world factors such as changes in viewpoint, lighting conditions, and cluttered backgrounds.

A practical solution to this problem must achieve the following:

- **Robustness to variation:** The system should handle moderate changes in viewpoint, illumination variation, and background clutter. It should also cope with partial occlusions and distortions that are common in real-world environments.
- **Accuracy and precision:** The alignment must be precise enough to support downstream measurement tasks, such as detecting dimensional deviations or ensuring the part fits within specified tolerances.
- **Compute efficiency:** The method must operate efficiently under constrained compute resources. For industrial deployment, it is important that the system can run in real-time or near real-time, with minimal computational overhead.
- **Measurable performance metrics:** The system should provide quantifiable metrics such as alignment error and success rate, allowing for objective performance evaluation.

1.3 Contributions

This work presents a novel, feature-based alignment method that satisfies the aforementioned requirements and demonstrates its applicability in template-to-photo alignment tasks. The contributions of this work are as follows:

- **A complete, modular implementation:** We provide a full implementation of the alignment pipeline with well-defined stages and outputs. The system is built with reproducibility in mind, ensuring that all results can be independently verified.
- **A robust alignment method:** The proposed method leverages feature matching and homography estimation to compute the geometric transformation between the template and photograph. It is designed to be robust to noise, variations in lighting, and partial occlusions.
- **Evaluation protocol:** We define a rigorous evaluation protocol with both quantitative and qualitative analyses. We use well-established metrics such as mean/median reprojection error and success rate, and we provide failure-case diagnostics to analyze the limits of the system.
- **Reproducibility artifacts:** All results, including metrics CSVs and generated figures, are available for download, enabling others to reproduce the reported findings and build upon this work. The full source code and dataset split details are included for transparency.

This report not only contributes to the field of image alignment but also provides insights into the challenges and solutions that arise when aligning template images to real-world photographs under varying conditions.

2 Notation and Definitions

This section defines symbols and terminology used throughout the report for both components: (i) VAEs (baseline and the referenced enhanced-capacity/hierarchical variant) on Dynamic MNIST, and (ii) conditional WGAN-GP for data augmentation on FashionMNIST.

2.1 Symbols

Data and distributions (VAE).

- $x \in \{0,1\}^D$: observed image (Dynamic MNIST uses dynamic binarization; $D = 28 \times 28$).
- $z \in \mathbb{R}^d$: continuous latent variable; d is the latent dimensionality.
- $p(z)$: latent prior (baseline uses $p(z) = \mathcal{N}(0, I)$).
- $q_\phi(z \mid x)$: encoder / variational posterior parameterized by ϕ.
- $p_\theta(x \mid z)$: decoder / likelihood parameterized by θ.
- $\mathcal{L}_{\text{ELBO}}(x)$: evidence lower bound (ELBO).
- $\text{KL}(q\|p)$: Kullback–Leibler divergence.
- $\mathcal{L}_{\text{rec}}(x) = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]$: reconstruction term (negative expected log-likelihood).
- $\mathcal{L}_{\text{KL}}(x) = \text{KL}(q_\phi(z \mid x) \,\|\, p(z))$: KL regularizer term (baseline).

Enhanced-capacity prior / pseudo-inputs (referenced VAE).

- $\{u_k\}_{k=1}^K$: pseudo-inputs (learnable parameters in input space or outputs of a pseudo-input network).

- $p_\lambda(z) = \frac{1}{K}\sum_{k=1}^{K} q_\phi(z \mid u_k)$: learned mixture prior with parameters λ (through $\{u_k\}$).
- $\mathcal{L}_{\mathrm{KL}}^{(\mathrm{mix})}(x) = \mathrm{KL}(q_\phi(z \mid x) \,\|\, p_\lambda(z))$: KL term against the learned prior.

Hierarchical VAE (two-layer latent).

- $z_2 \in \mathbb{R}^{d_2}$: top-level latent variable.
- $z_1 \in \mathbb{R}^{d_1}$: bottom-level latent variable.
- $p_\theta(z_1 \mid z_2)$: conditional prior (top-down) in the hierarchical decoder.
- $q_\phi(z_2 \mid x)$, $q_\phi(z_1 \mid x, z_2)$: hierarchical inference factors.

Likelihood estimation.

- K_{IS}: number of importance samples for likelihood estimation.
- $\widehat{\log p_\theta(x)}$: importance-weighted estimate of log-likelihood (IWAE-style).

Conditional WGAN-GP (GAN augmentation).

- $y \in \{1, \ldots, C\}$: class label ($C = 10$ for FashionMNIST).
- $z \sim \mathcal{N}(0, I)$: generator noise vector.
- $G_\psi(z, y)$: conditional generator parameterized by ψ.
- $D_\omega(x, y)$: conditional critic/discriminator parameterized by ω (projection discriminator).
- $e_G(y) \in \mathbb{R}^{d_y}$: generator label embedding; $e_D(y) \in \mathbb{R}^{d_f}$: critic label embedding.
- $\phi_\omega(x) \in \mathbb{R}^{d_f}$: critic feature representation used in projection discrimination.
- λ_{GP}: gradient penalty coefficient (assignment specifies $\lambda_{\mathrm{GP}} = 10$).
- $\epsilon_{\mathrm{drift}}$: drift penalty coefficient (assignment specifies 0.001).
- λ_{emb}: embedding L_2 regularization coefficient (assignment specifies 0.001).
- N_{critic}: number of critic updates per generator update (assignment specifies 5).
- RUN: total training steps (assignment specifies 4000).

2.2 Abbreviations

- VAE: Variational Autoencoder.
- ELBO: Evidence Lower Bound.
- KL: Kullback–Leibler divergence.
- IWAE: Importance Weighted Autoencoder estimator (importance sampling for likelihood).
- cGAN: Conditional GAN.
- WGAN-GP: Wasserstein GAN with Gradient Penalty.

- SN: Spectral Normalization.
- SA: Self-Attention (spatial attention).
- CA: Channel Attention (Squeeze-and-Excitation style).
- PD: Projection Discriminator.

2.3 Key Modeling Assumptions

- **Dynamic MNIST observation model:** pixels are modeled as conditionally independent Bernoulli variables given latents ($p_\theta(x \mid z)$ Bernoulli), consistent with the dynamic binarization protocol.
- **Variational inference:** $q_\phi(z \mid x)$ (and its hierarchical factors) are Gaussian with diagonal covariance, enabling reparameterization-based training.
- **WGAN-GP Lipschitz control:** the critic is approximately 1-Lipschitz via gradient penalty (and SN on selected layers), which is required for the Wasserstein objective to be well-behaved.
- **Augmentation validity:** synthetic samples are used only to expand the training distribution; generalization is judged exclusively on held-out real test data.

3 System Overview

3.1 Pipeline Summary

The alignment system described in this report follows a standard feature-based registration pipeline, which processes the template and photograph in a sequence of deterministic stages to achieve the desired template-to-photo overlay. The overall steps are as follows:

1. **Preprocessing of Images:** Both the template image ($\mathcal{I}_T$) and the query photograph ($\mathcal{I}_Q$) are preprocessed. This typically involves converting the images to grayscale for simplicity and ensuring they are normalized for consistent feature extraction. In some cases, contrast normalization (e.g., CLAHE) is applied to enhance keypoint detectability, especially when dealing with low-contrast images.
2. **Keypoint Detection and Description:** Local features are detected using the Oriented FAST and Rotated BRIEF (ORB) algorithm [**?**]. ORB is chosen for its computational efficiency and its robustness to rotation and scale variations in the images. The detected keypoints are described using binary descriptors, which are efficient for matching and computationally lightweight.
3. **Descriptor Matching and Filtering:** Once the descriptors are extracted, they are matched between the template and the query image. Hamming distance is used to measure the similarity between descriptors. A cross-checking procedure is applied to reduce incorrect matches, and a distance threshold is used to filter out weak matches, ensuring only the most reliable correspondences are retained.

4. **Homography Estimation via RANSAC:** The next step is to estimate the homography **H**, which models the projective transformation between the template and the query photograph. The homography is computed using RANSAC [**?**], a robust estimator that minimizes the effect of outlier matches by iteratively fitting the model to the data and retaining only the inliers that are consistent with the estimated homography.

5. **Template Warping and Overlay Generation:** After estimating the homography, the template is warped into the coordinate frame of the photograph using the computed homography **H**. The warped template is then blended with the query photograph to create the final overlay image for inspection.

6. **Metrics Export and Analysis:** Quantitative performance metrics, such as reprojection error, success rate, and runtime, are computed and exported. Additionally, failure-case examples are identified and stored to provide insights into boundary conditions and potential areas for improvement in the system.

3.2 Repository Layout (Expected)

To maintain organization and clarity, the repository is structured with clear directory divisions for each component. Below is the expected folder layout, which helps manage data, scripts, and results:

```
report/
  main.tex          % Main report LaTeX file
  *.tex             % Other LaTeX files (e.g., for individual sections)
  references.bib    % BibTeX file containing references
src/
  (your python modules) % Python scripts for the alignment pipeline
data/
  templates/        % Folder containing the template images
  photos/           % Folder containing the query photographs
  annotations/      % (optional) Folder for ground-truth point correspondences or masks
results/
  overlays/         % Folder for saving overlay images (template overlaid on query images)
  figures/          % Folder for figures used in the report (e.g., error distributions, runt
  metrics/          % Folder for quantitative metrics (CSV/JSON files)
  matches/          % (optional) Folder for debug outputs showing raw matches or intermediat
```

This structure ensures that each component of the project–data, code, and results–are clearly separated, facilitating both reproducibility and further experimentation.

3.3 Why Feature-Based Registration

Feature-based registration is a widely adopted method for aligning images, particularly when dealing with planar scenes or objects where the relationship between the template and the query

image can be approximated by a projective transformation (homography). This approach has several advantages:

- **Computational Efficiency:** Feature-based methods like ORB provide a good balance between accuracy and computational efficiency. ORB's binary descriptors are lightweight and allow for fast matching, which is essential for real-time applications or large-scale datasets.
- **No Training Data Required:** Unlike deep learning-based methods, feature-based registration does not require large datasets for training. This makes it a particularly appealing choice when labeled data is scarce or when the method needs to be applied to a wide variety of images without re-training.
- **Robustness to Moderate Viewpoint Changes:** ORB features are invariant to rotation and scale, making the method robust to moderate viewpoint changes and ensuring that keypoints remain detectable even when the template and photograph are viewed from different angles or distances.
- **Low Latency:** The method operates with low latency, making it suitable for applications that require fast processing, such as interactive inspection systems or on-the-fly part verification during assembly lines.

Feature-based registration methods are particularly effective when there is sufficient texture or structure in the images for feature detection. Under moderate viewpoint changes, these methods can yield high-quality alignments with low computational overhead [**? ?**]. However, they may struggle in the presence of repetitive textures, glare, or occlusion, which can be mitigated by applying appropriate preprocessing and matching strategies.

By focusing on feature-based methods, this work provides a solution that is both efficient and flexible, making it applicable to a wide range of practical scenarios in industrial inspection, robotic assembly, and other areas requiring template-to-photo alignment.

4 Related Work

Generative modeling has two dominant paradigms relevant to this report: likelihood-based latent-variable models (VAEs) and adversarial models (GANs). Both have extensive literature on objectives, stability, and failure modes, particularly under limited data or highly expressive decoders.

4.1 Variational Autoencoders and objective design

VAEs optimize a variational lower bound on $\log p_\theta(x)$, enabling scalable amortized inference via an encoder network [1, 2]. While VAEs provide a principled likelihood-based framework, empirical work has documented limitations including blurry samples (under simple likelihoods) and *posterior collapse*, where the KL term approaches zero and the latent variable becomes uninformative, especially with expressive decoders.

A major line of research improves VAE performance by changing the objective or increasing representational capacity. Importance-weighted objectives provide tighter bounds and can im-

prove training and evaluation by using multiple importance samples [3]. Other approaches alter priors (e.g., richer latent distributions) or introduce hierarchical latents, enabling multimodal structure and better latent utilization. The referenced method in this assignment follows this direction by introducing a modified loss/prior mechanism and a hierarchical architecture, with the explicit goal of increasing capacity and mitigating collapse-like behavior.

4.2 Likelihood estimation for latent-variable models

Exact likelihood computation for latent-variable models is generally intractable due to the integral over z. Importance sampling (IWAE-style estimators) is a standard approach for estimating $\log p_\theta(x)$ in a controlled and comparable way across models [3]. This report uses a consistent estimator and sample budget across VAE variants to ensure fair comparisons.

4.3 Conditional GANs, stability, and augmentation

GANs generate samples by adversarial training and often yield sharper samples than likelihood-based models, but training can be unstable and sensitive to architecture and regularization. Wasserstein GAN objectives improve gradient behavior by approximating Earth Mover distance, and WGAN-GP enforces the Lipschitz constraint via a gradient penalty, substantially improving stability in practice [4]. Spectral normalization further stabilizes training by constraining the discriminator's operator norm [5].

For conditional generation, projection discriminators incorporate label conditioning through inner products in feature space, improving class-conditional fidelity and stability relative to naive concatenation [6]. Attention mechanisms have also proven effective in generative modeling by enabling long-range spatial interactions (self-attention) and adaptive channel reweighting (channel attention, commonly formalized as squeeze-and-excitation) [7, 8].

Finally, data augmentation is a standard remedy for limited training data in supervised learning. In this report, GAN-based augmentation is evaluated by a controlled comparison between classifiers trained on the same limited real subset with and without synthetic samples, with all other training factors held fixed.

5 Methodology

5.1 Variational Autoencoders (VAE)

Latent-variable model and variational inference. We consider a latent-variable generative model

$$p_\theta(x, z) = p_\theta(x \mid z)\, p(z),$$

where $x \in \mathbb{R}^D$ denotes an observed image and $z \in \mathbb{R}^d$ is a continuous latent variable. Since the exact posterior $p_\theta(z \mid x)$ is typically intractable, we introduce an amortized variational approximation (encoder)

$$q_\phi(z \mid x).$$

For Dynamic MNIST, we use a Bernoulli observation model

$$p_\theta(x \mid z) = \prod_{i=1}^{D} \text{Bernoulli}(x_i; \pi_\theta(z)_i),$$

where $\pi_\theta(z) \in (0,1)^D$ is the decoder output after a sigmoid nonlinearity.

ELBO objective and KL regularization. Training proceeds by maximizing the evidence lower bound (ELBO) on the log-likelihood $\log p_\theta(x)$ [1, 2]:

$$\log p_\theta(x) \;\geq\; \mathcal{L}_{\text{ELBO}}(x) = \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right] - \text{KL}(q_\phi(z \mid x) \,\|\, p(z))\,.$$

The first term is a reconstruction (negative cross-entropy for Bernoulli pixels), while the KL divergence regularizes the approximate posterior toward the prior, controlling latent capacity and enabling sampling from $p(z)$ at test time.

Reparameterization trick. We parameterize $q_\phi(z \mid x)$ as a diagonal Gaussian,

$$q_\phi(z \mid x) = \mathcal{N}\Big(z; \mu_\phi(x), \text{diag}(\sigma^2_\phi(x))\Big)\,,$$

and apply the reparameterization trick to enable low-variance gradient estimation:

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

Enhanced-capacity objective: flexible prior via pseudo-inputs. A core limitation of standard VAEs is that a simple fixed prior (e.g., $\mathcal{N}(0, I)$) can over-regularize $q_\phi(z \mid x)$, encouraging inactive latent dimensions and contributing to posterior collapse, particularly when the decoder is expressive. Following the referenced method, we replace the fixed prior with a learnable, data-adaptive prior defined through *pseudo-inputs* $\{u_k\}_{k=1}^K$:

$$p_\lambda(z) = \frac{1}{K} \sum_{k=1}^{K} q_\phi(z \mid u_k),$$

where u_k are optimized parameters in input space (or are generated by a small network) and $q_\phi(z \mid u_k)$ reuses the encoder. This prior increases representational capacity by forming a multimodal latent distribution aligned with the aggregate posterior, thereby relaxing an overly restrictive KL constraint. The training objective becomes:

$$\mathcal{L}(x) = \mathbb{E}_{q_\phi(z|x)}\left[\log p_\theta(x \mid z)\right] - \text{KL}(q_\phi(z \mid x) \,\|\, p_\lambda(z))\,,$$

with the KL now computed against the learned mixture prior. In practice, this modification can improve latent utilization and sample quality by avoiding a mismatch between $q_\phi(z \mid x)$ and a simplistic prior.

Hierarchical latent architecture. To further increase capacity, we implement a hierarchical latent model with two stochastic layers (e.g., $z_2 \to z_1 \to x$), where the generative model factorizes as

$$p_\theta(x, z_1, z_2) = p_\theta(x \mid z_1)\, p_\theta(z_1 \mid z_2)\, p(z_2),$$

and the inference model factorizes as

$$q_\phi(z_1, z_2 \mid x) = q_\phi(z_2 \mid x)\, q_\phi(z_1 \mid x, z_2).$$

The corresponding ELBO is

$$\mathcal{L}_{\text{hier}}(x) = \mathbb{E}_{q_\phi(z_1,z_2|x)}\left[\log p_\theta(x \mid z_1)\right] - \mathbb{E}_{q_\phi(z_2|x)}\left[\mathrm{KL}(q_\phi(z_1 \mid x, z_2) \,\|\, p_\theta(z_1 \mid z_2))\right] - \mathrm{KL}(q_\phi(z_2 \mid x) \,\|\, p(z_2)),$$

where the top-level prior $p(z_2)$ may be chosen as a standard normal or replaced by the pseudo-input prior $p_\lambda(z_2)$, depending on the assignment's specification.

Likelihood estimation. Since $\log p_\theta(x) = \log \int p_\theta(x \mid z) p(z)\, dz$ is not directly tractable, we estimate log-likelihood using an importance-weighted (IWAE-style) estimator [3]. For K importance samples $z^{(k)} \sim q_\phi(z \mid x)$,

$$\log p_\theta(x) \approx \log \left(\frac{1}{K} \sum_{k=1}^{K} \frac{p_\theta(x \mid z^{(k)})\, p(z^{(k)})}{q_\phi(z^{(k)} \mid x)} \right).$$

Diagnostics for posterior behavior and collapse. To analyze latent utilization, we report (i) KL contributions per dimension (or per group in hierarchical models), (ii) active units (e.g., dimensions with $\mathrm{Var}_x[\mathbb{E}(z \mid x)]$ above a threshold), and (iii) aggregate posterior statistics compared with the prior. We complement these with latent traversals and PCA of inferred codes to test whether principal directions correspond to meaningful factors of variation.

5.2 Conditional GAN for Data Augmentation (WGAN-GP)

Problem setting and data splits. We consider FashionMNIST under a limited-data regime. The training set is partitioned into two *non-overlapping* subsets: one for GAN training and one for classifier training. The GAN is used to generate additional labeled samples for augmentation; the downstream classifier is trained on either real-only or real+synthetic data and evaluated on a held-out test set.

Conditional generator. The generator is conditional on class label $y \in \{1, \ldots, C\}$ and noise $z \sim \mathcal{N}(0, I)$. A learnable embedding $e_G(y) \in \mathbb{R}^{d_y}$ is concatenated with z to form the generator input:

$$\tilde{z} = [z;\, e_G(y)], \qquad x_{\text{fake}} = G_\psi(\tilde{z}, y).$$

The architecture follows the assignment's specified ConvTranspose/upsampling blocks, including attention modules at designated resolutions.

Projection discriminator (conditional critic). The discriminator (critic) follows the projection discriminator formulation [6]. Let $\phi_\omega(x) \in \mathbb{R}^{d_f}$ denote the discriminator feature vector before the final output layer, and let $e_D(y) \in \mathbb{R}^{d_f}$ be the label embedding in the discriminator. The conditional score is computed as

$$D_\omega(x, y) = s_\omega(\phi_\omega(x)) + \langle \phi_\omega(x),\, e_D(y) \rangle,$$

where $s_\omega(\cdot)$ is a learned scalar head and $\langle \cdot, \cdot \rangle$ denotes an inner product. This construction enforces class-conditional discrimination without explicitly concatenating labels to image channels.

Spectral normalization. To stabilize adversarial training, we apply spectral normalization (SN) to convolutional (and transposed convolutional) layers [5]. SN constrains each layer's operator norm by normalizing its weight matrix W via its largest singular value $\sigma_{\max}(W)$:

$$\bar{W} = \frac{W}{\sigma_{\max}(W)},$$

which controls the Lipschitz constant of the discriminator and improves training stability.

Attention mechanisms. We incorporate two attention modules, following the assignment definitions.

Self-attention (as in SAGAN [7]) captures long-range spatial dependencies. Given feature maps $x \in \mathbb{R}^{B\times C\times H\times W}$ with $N = HW$, we compute projections f, g, h and attention weights $\beta \in \mathbb{R}^{B\times N\times N}$:

$$\beta = \mathrm{softmax}(f^\top g),$$

and produce an output o aggregated from h (and then mapped through v), followed by a residual connection with a learnable scalar γ:

$$y = \gamma o + x.$$

Channel attention (SE-style [8]) recalibrates channel-wise responses. Global average pooling produces a channel descriptor $c \in \mathbb{R}^{B\times C}$, which is passed through two fully-connected layers with ReLU and a sigmoid gate to generate channel weights that modulate the original feature map.

WGAN-GP objective with regularization. We use the Wasserstein GAN objective with gradient penalty (WGAN-GP) [4]. The critic aims to maximize the Wasserstein estimate:

$$\mathcal{W} \approx \mathbb{E}[D(x_{\text{real}}, y)] - \mathbb{E}[D(x_{\text{fake}}, y)],$$

subject to a soft 1-Lipschitz constraint enforced by the gradient penalty:

$$\mathcal{L}_{\text{GP}} = \lambda_{\text{GP}} \mathbb{E}_{\hat{x}} \left(\|\nabla_{\hat{x}} D(\hat{x}, y)\|_2 - 1 \right)^2, \qquad \lambda_{\text{GP}} = 10,$$

where $\hat{x} = \epsilon x_{\text{real}} + (1-\epsilon) x_{\text{fake}}$ and $\epsilon \sim \mathcal{U}(0, 1)$.

In addition, we incorporate: (i) a drift penalty to prevent critic outputs from drifting:

$$\mathcal{L}_{\text{drift}} = \epsilon_{\text{drift}}\, \mathbb{E}\left[D(x_{\text{real}}, y)^2\right], \qquad \epsilon_{\text{drift}} = 0.001,$$

and (ii) discriminator embedding regularization:

$$\mathcal{L}_{\text{emb}} = \lambda_{\text{emb}}\, \|e_D\|_2^2, \qquad \lambda_{\text{emb}} = 0.001.$$

The discriminator loss (to minimize) is therefore:

$$\mathcal{L}_D = -\Big(\mathbb{E}[D(x_{\text{real}}, y)] - \mathbb{E}[D(x_{\text{fake}}, y)]\Big) + \mathcal{L}_{\text{GP}} + \mathcal{L}_{\text{drift}} + \mathcal{L}_{\text{emb}},$$

and the generator loss is:

$$\mathcal{L}_G = -\mathbb{E}[D(x_{\text{fake}}, y)].$$

Optimization and training protocol. We use Adam optimizers with the assignment's hyperparameters: learning rates $\text{LR}(G) = 2 \times 10^{-4}$ and $\text{LR}(D) = 4 \times 10^{-4}$, $\beta_1 = 0.0$, $\beta_2 = 0.9$, and an exponential decay scheduler with $\gamma = 0.95$ applied every 100 steps. Weight decay is grouped such that embedding parameters receive decay 10^{-3} and other parameters receive zero decay, matching the specification. We update the discriminator $N_{\text{critic}} = 5$ times per generator update, detach fake samples during critic updates, and log the Wasserstein estimate, gradient penalty, drift, embedding regularization, and total losses at each step.

Sampling protocol and augmentation evaluation. To monitor training progression, we generate class-conditional grids at fixed steps using fixed noise and fixed labels (covering all classes), enabling controlled qualitative comparisons across time. For augmentation, we generate a specified number of synthetic samples per class and train a classifier under two regimes: real-only and real+synthetic. We then compare overall accuracy, class-wise performance, and error patterns to quantify whether generative augmentation improves generalization in the limited data setting.

6 Data and Experimental Protocol

7 Datasets and Data Loading

7.1 Datasets

This report uses two standard vision datasets, chosen to match the two tasks in the assignment: (i) likelihood-based generative modeling and latent analysis (Dynamic MNIST), and (ii) conditional generation for augmentation under limited labeled data (FashionMNIST).

All experiments use the canonical 28×28 single-channel (grayscale) image format. For computational reproducibility, we fix preprocessing and data-loading conventions (normalization range, batching policy, and optional subset selection) and log the exact settings used in each run.

7.1.1 Dynamic MNIST

Dynamic MNIST is derived from the MNIST handwritten digits dataset [**?**]. In contrast to a *fixed* binarization (where each grayscale image is thresholded once and stored as a static binary image), Dynamic MNIST uses *stochastic binarization* at each presentation. Concretely, let $x^{\text{gray}} \in [0,1]^D$ denote the normalized grayscale image (flattened to $D = 784$ pixels). Each training step generates a binary sample $x \in \{0,1\}^D$ via pixel-wise Bernoulli sampling:

$$x_i \sim \text{Bernoulli}(x_i^{\text{gray}}), \quad i = 1, \ldots, D.$$

This yields a distribution over binary images per underlying grayscale digit and is widely used in likelihood-based generative modeling because it avoids treating an arbitrary single thresholding as ground truth. In this report, Dynamic MNIST is used to evaluate:

- the baseline VAE objective vs. the modified objective and hierarchical structure required by the assignment,
- low-dimensional vs. high-dimensional latent settings,
- posterior behavior (KL diagnostics, latent activity, collapse indicators),
- log-likelihood estimation using importance-weighted estimators.

Important implementation note (dynamic vs. static). If the data pipeline loads only fixed images without Bernoulli resampling, the dataset becomes *static* (deterministic) and no longer matches the Dynamic MNIST protocol. Therefore, Dynamic MNIST must apply the Bernoulli sampling either as a dataset transform or inside the training loop before computing the reconstruction loss.

7.1.2 FashionMNIST

FashionMNIST contains 28×28 grayscale images from 10 clothing categories (e.g., T-shirt/top, trouser, pullover) [**?**]. It is used for class-conditional generation and augmentation because it supports clear label conditioning and direct measurement of downstream classification generalization.

To match the GAN specification, inputs are normalized to $[-1, 1]$:

$$x \leftarrow \frac{x - 0.5}{0.5}.$$

This normalization is consistent with a generator output activation $\tanh(\cdot)$ producing samples in $[-1, 1]$.

7.2 Data Loading and Preprocessing Implementation (`deepgen/data.py`)

This project uses a minimal, auditable PyTorch data-loading implementation. The file `deepgen/data.py` provides two core utilities: (i) a consistent transform stack for MNIST-like datasets, and (ii) train/test `DataLoader` construction with controlled batching behavior.

7.2.1 Transform definition: `mnist_transforms`

The function `mnist_transforms(scale_to_minus1_1: bool)` returns a composed transform:

- **`transforms.ToTensor()`:** converts a PIL image to a torch tensor in $[0, 1]$, with shape $(1, 28, 28)$.
- **`transforms.Normalize(mean=(0.5,), std=(0.5,))`:** if enabled, maps the tensor from $[0, 1]$ to $[-1, 1]$ via $x \mapsto (x - 0.5)/0.5$.

Scientific rationale.

- For GAN training, $[-1, 1]$ scaling is standard when the generator output uses tanh, improving numerical stability and making the data range consistent with model outputs.
- For VAE training, the reconstruction likelihood for Bernoulli decoders typically assumes targets in $[0, 1]$. If the pipeline stores inputs in $[-1, 1]$, the reconstruction target must be converted back to $[0, 1]$ (as done in the VAE loss implementation by mapping $(x + 1)/2$).

7.2.2 Loader construction: `get_mnist_loaders`

The function `get_mnist_loaders(root, batch_size, num_workers, train_subset, scale_to_minus1_1, pin_memory)` constructs two loaders (train and test), using the same transform stack:

Dataset objects.

- **Training set:** `datasets.MNIST(train=True, download=True, transform=tfm)`.
- **Test set:** `datasets.MNIST(train=False, download=True, transform=tfm)`.

Optional subset selection (`train_subset`). If `train_subset` is provided, the code wraps the training dataset in `torch.utils.data.Subset` using the first `train_subset` indices. This feature is intended for:

- rapid debugging (short runs to validate losses, shapes, and logging),
- ablations under limited data regimes,
- controlled experiments where dataset size is an independent variable.

The implementation explicitly checks `train_subset > 0` to prevent silent empty datasets.

DataLoader policy (train). The training `DataLoader` is created with:

- **`shuffle=True`:** ensures SGD sees randomized mini-batches.
- **`drop_last=True`:** enforces fixed batch size, which simplifies: (i) batch-norm statistics consistency, (ii) logging alignment across steps, (iii) gradient penalty code (in GANs) that often assumes stable batch shapes.
- **`pin_memory`:** enabled only when CUDA is available, improving host-to-device transfer throughput.

- **num_workers:** controls parallel data loading; this can affect throughput but should not affect correctness when determinism controls are used (see **??**).

DataLoader policy (test). The test `DataLoader` uses:

- **shuffle=False:** deterministic evaluation order.
- **drop_last=False:** preserves all test samples to avoid biased evaluation.

7.2.3 How Dynamic MNIST fits this implementation

The provided `deepgen/data.py` pipeline loads MNIST images as continuous tensors (in $[0, 1]$ after `ToTensor`, optionally mapped to $[-1, 1]$). To obtain **Dynamic MNIST**, a Bernoulli sampling step must be applied *per iteration*. Scientifically, this can be implemented in one of two equivalent ways:

Option A (transform-level dynamic binarization). Insert a stochastic transform between `ToTensor` and `Normalize`. This ensures each `__getitem__` produces a newly sampled binary image.

Option B (training-loop dynamic binarization). Keep the dataset in grayscale, and at each step sample:

$$x \leftarrow \mathbf{1}\{u < x^{\text{gray}}\}, \quad u \sim \text{Uniform}(0, 1),$$

then (optionally) rescale to match the model input convention. This approach makes the stochasticity explicit in the training procedure and is often preferred when one wants to log both the underlying grayscale and sampled binary realizations.

Consistency with the VAE likelihood model. If the decoder likelihood is Bernoulli (binary cross-entropy reconstruction), then the training target must be in $[0, 1]$ and interpreted as a binary sample (or a Bernoulli parameterization if using continuous relaxation). Therefore, when the input tensor is stored in $[-1, 1]$ for shared code reuse, the VAE loss must map it back to $[0, 1]$ before BCE, which is exactly why the VAE code uses:

$$x_{01} = (x + 1)/2.$$

7.3 Limited-data regime and non-overlapping splits (FashionMNIST)

To simulate limited labeled data and ensure scientifically valid augmentation evaluation, we construct two *non-overlapping* subsets from the FashionMNIST training set [**?**]:

- $\mathcal{D}_{\text{GAN}}$: used exclusively to train the conditional WGAN-GP generator/critic.
- $\mathcal{D}_{\text{CLS}}$: used exclusively to train the downstream classifier.

Both subsets are stratified by class. The per-class sample count follows the assignment specification (e.g., N samples per class). We persist the selected indices to disk (e.g., `data/splits/gan_idx.pt`

and `data/splits/cls_idx.pt`) and reuse them for all runs. This prevents accidental leakage:

$$\mathcal{D}_{\text{GAN}} \cap \mathcal{D}_{\text{CLS}} = \emptyset.$$

This constraint is essential; if overlap exists, the generator can memorize examples that the classifier sees during training, and the "augmentation" would not represent an independent distributional expansion.

7.4 Train/Validation/Test protocol

VAE (Dynamic MNIST). Models are trained on the training split. Evaluation uses the held-out test split for: (i) ELBO decomposition and KL diagnostics, (ii) importance-weighted log-likelihood estimates, (iii) latent analysis (posterior statistics, PCA/traversals), (iv) qualitative sampling. If a validation set is used for hyperparameter selection, it is drawn only from the training split to preserve test integrity.

GAN augmentation (FashionMNIST).

- The GAN is trained only on $\mathcal{D}_{\text{GAN}}$.
- The classifier is trained on $\mathcal{D}_{\text{CLS}}$ (real-only baseline) or on $\mathcal{D}_{\text{CLS}}$ plus synthetic samples generated by the trained GAN (augmented condition).
- Final evaluation uses the untouched FashionMNIST test set containing only real samples.

7.5 Dataset integrity checks

Before training, we perform and log:

- class balance verification in $\mathcal{D}_{\text{GAN}}$ and $\mathcal{D}_{\text{CLS}}$,
- explicit confirmation of zero overlap between splits (intersection cardinality equals zero),
- normalization range checks (min/max; optionally histograms) to verify $[0, 1]$ or $[-1, 1]$ as intended,
- visual sanity checks after preprocessing (random grids) to detect transform mistakes.

These checks mitigate common implementation errors that can invalidate generative model comparisons or augmentation conclusions.

7.6 Preprocessing Goals

Preprocessing is designed to (i) match the probabilistic assumptions of each generative model, (ii) remove avoidable nuisance variation, and (iii) ensure that comparisons across models are scientifically fair (identical data transformations, consistent splits, and identical evaluation pipelines).

Concretely, the preprocessing pipeline must satisfy the following:

- **Likelihood consistency (VAE):** the input representation must match the decoder likelihood family (Bernoulli for Dynamic MNIST).

- **Scale consistency (GAN):** real images must be normalized to the same numeric range as the generator output (typically $[-1, 1]$ with tanh output).
- **Split integrity (augmentation study):** the GAN training subset and the classifier training subset must be disjoint, with indices logged and reused.
- **Reproducibility:** all stochastic elements (dynamic binarization, sampling indices) must be controlled via fixed seeds and saved split files.

7.7 Preprocessing for Dynamic MNIST (VAE)

7.7.1 Dynamic binarization protocol

Dynamic MNIST is formed from grayscale MNIST by sampling a Bernoulli variable independently at each pixel using the grayscale intensity as the Bernoulli parameter. If $x^{\text{gray}} \in [0, 1]^D$ is the original normalized grayscale image, then each training presentation uses:

$$x_i \sim \text{Bernoulli}\left(x_i^{\text{gray}}\right), \qquad i = 1, \ldots, D.$$

This is repeated *on-the-fly* during training (and typically also during evaluation for consistency, unless the assignment specifies a fixed binarized test set). Dynamic binarization reduces sensitivity to a single arbitrary thresholding and is standard in likelihood-oriented evaluation of VAEs [1, 2].

7.7.2 Model-aligned input and decoder likelihood

Because $x \in \{0, 1\}^D$ under dynamic binarization, we use a Bernoulli observation model:

$$p_\theta(x \mid z) = \prod_{i=1}^{D} \text{Bernoulli}\left(x_i; \pi_\theta(z)_i\right),$$

where $\pi_\theta(z) \in (0, 1)^D$ is the decoder output after sigmoid. The corresponding reconstruction term is the negative binary cross-entropy. This alignment is essential: mismatching preprocessing (e.g., continuous-valued pixels) with a Bernoulli likelihood produces invalid likelihood comparisons.

7.7.3 Practical implementation details

- **Input scaling:** convert raw uint8 MNIST images to float in $[0, 1]$ before sampling.
- **Stochasticity control:** if strict reproducibility is required, fix the RNG seed per epoch and log it; otherwise, keep the seed fixed globally but allow per-iteration variability (explicitly stated in the report).
- **Evaluation consistency:** use the *same* binarization convention for baseline and proposed VAEs (and across latent dimensionalities) to avoid confounding.

7.8 Preprocessing for FashionMNIST (GAN augmentation)

7.8.1 Normalization and numeric range

FashionMNIST images are transformed using the assignment's normalization:

$$x \leftarrow \frac{x - 0.5}{0.5},$$

which maps $x \in [0, 1]$ to $x \in [-1, 1]$. This is compatible with a generator using tanh at the output layer. The transform is implemented as:

```
transforms.ToTensor()
transforms.Normalize(mean=(0.5,), std=(0.5,))
```

7.8.2 Label handling

Labels are kept as integer class IDs $y \in \{0, \ldots, 9\}$ and embedded in the generator and discriminator as specified (embedding dimension per assignment). No label smoothing is used unless explicitly required.

7.8.3 Ensuring fairness in augmentation evaluation

The classifier receives either:

- **Real-only:** limited classifier subset,
- **Real + synthetic:** the same real subset plus generated samples.

All other factors (classifier architecture, optimizer, training schedule, and evaluation set) are held constant to isolate the causal effect of augmentation.

7.9 Scientific Rationale and Controlled Variables

Preprocessing choices directly affect the learned distribution and the validity of model comparisons:

- **Dynamic binarization** changes the effective data distribution; therefore, it must be treated as part of the experimental definition and not an implementation detail.
- **Normalization** changes gradient scales and critic behavior in WGAN-GP; mismatched normalization can lead to unstable training or misleading comparisons across runs.
- **Split integrity** is central: if GAN and classifier subsets overlap, augmentation evaluation becomes invalid because the generator may memorize examples the classifier later sees as "augmented" samples.

Accordingly, preprocessing settings and split indices are recorded in configuration files and reported explicitly in the experiment tables.

7.10 Variational Autoencoders (VAE)

Latent-variable generative model. We model an image $x \in \mathbb{R}^D$ using a continuous latent variable $z \in \mathbb{R}^d$ with parameters θ:

$$p_\theta(x, z) = p_\theta(x \mid z)\, p(z).$$

Learning requires the marginal likelihood $\log p_\theta(x) = \log \int p_\theta(x \mid z) p(z)\, dz$, which is generally intractable for neural decoders. We therefore introduce an amortized inference model (encoder) $q_\phi(z \mid x)$ and optimize a variational bound [1, 2].

Observation model and data range conventions (critical for correctness). In our implementation (`deepgen/models/vae.py`), the decoder outputs $x_{\text{rec}} \in (0,1)^D$ via a sigmoid. Consequently, reconstruction is evaluated with a Bernoulli/BCE-style likelihood:

$$p_\theta(x \mid z) = \prod_{i=1}^{D} \text{Bernoulli}(x_i;\, \pi_\theta(z)_i)\,, \qquad \pi_\theta(z) = \sigma(f_\theta(z)).$$

The training input x is stored in $[-1, 1]$ space (consistent with `Normalize(0.5,0.5)`), but BCE requires targets in $[0, 1]$. Therefore, the implementation converts targets as

$$x_{01} = \frac{x + 1}{2},$$

and uses $\text{BCE}(x_{\text{rec}}, x_{01})$ summed over pixels (see `elbo_loss`).

ELBO objective and its decomposition. The evidence lower bound (ELBO) satisfies:

$$\log p_\theta(x) \geq \mathcal{L}_{\text{ELBO}}(x) = \mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x \mid z)\big] - \text{KL}(q_\phi(z \mid x) \,\|\, p(z))\,.$$

The first term is a reconstruction term (negative cross-entropy for Bernoulli pixels), and the KL term regularizes the approximate posterior toward the prior, controlling latent capacity and enabling sampling.

Encoder parameterization and the reparameterization trick. We take $q_\phi(z \mid x)$ to be a diagonal Gaussian:

$$q_\phi(z \mid x) = \mathcal{N}\Big(z; \mu_\phi(x), \text{diag}(\sigma_\phi^2(x))\Big)\,,$$

with

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I),$$

which yields low-variance gradient estimates and is implemented as `reparameterize(mu, logvar)`.

Architecture summary (what is actually implemented). The VAE encoder uses a convolutional stem followed by residual blocks with optional squeeze-and-excitation (SE) gating:

$$28\times28 \xrightarrow{\text{Conv}} 28\times28 \xrightarrow{\text{ResDown}} 14\times14 \xrightarrow{\text{ResDown}} 7\times7 \xrightarrow{\text{Res}} 7\times7 \xrightarrow{\text{FC}} (\mu, \log\sigma^2).$$

The decoder maps z through two fully-connected layers to a 7×7 feature map, then upsamples twice back to 28×28 via transposed convolutions, ending with a sigmoid output layer.

Flexible prior via pseudo-inputs (VampPrior) and why it matters. A frequent failure mode of VAEs with expressive decoders is *over-regularization* toward a simplistic prior (e.g., $\mathcal{N}(0, I)$), which can yield inactive latent dimensions and posterior collapse. To increase prior flexibility, we use a VampPrior [9]:

$$p_\lambda(z) = \frac{1}{K}\sum_{k=1}^{K} q_\phi(z \mid u_k),$$

where $\{u_k\}_{k=1}^{K}$ are learnable pseudo-inputs. In the implementation, pseudo-inputs are direct image-shaped parameters (`self.pseudos`) and are passed through `tanh` to keep them approximately in $[-1, 1]$ (consistent with the encoder's expected input range).

KL term with a mixture prior (how we compute it in practice). When $p(z)$ is a mixture defined via $q_\phi(z \mid u_k)$, the KL term is computed via Monte Carlo:

$$\mathrm{KL}(q_\phi(z \mid x) \,\|\, p_\lambda(z)) = \mathbb{E}_{q_\phi(z|x)}[\log q_\phi(z \mid x) - \log p_\lambda(z)]\,.$$

The code implements:

$$\log p_\lambda(z) = \log \frac{1}{K}\sum_{k=1}^{K} q_\phi(z \mid u_k) = \mathrm{logsumexp}_k\big(\log q_\phi(z \mid u_k)\big) - \log K,$$

computed with numerically stable `logsumexp`. This is the central place where VampPrior increases capacity: it relaxes the mismatch between the aggregate posterior and a fixed unimodal prior.

β-weighting and reporting. For controlled latent capacity, we optionally use a β coefficient:

$$\mathcal{L}(x) = \mathbb{E}_{q_\phi(z|x)}[-\log p_\theta(x \mid z)] + \beta \cdot \mathrm{KL}(q_\phi(z \mid x) \,\|\, p_\lambda(z))\,,$$

which matches the signature `elbo_loss(..., beta)`. In results, we report total loss, reconstruction term, and KL term separately to diagnose whether improvements come from fidelity (recon) or representation (KL).

Likelihood estimation (IWAE-style). Because ELBO is a lower bound, we estimate $\log p_\theta(x)$ using an importance-weighted estimator [3]:

$$\log p_\theta(x) \approx \log\left(\frac{1}{K}\sum_{k=1}^{K}\frac{p_\theta(x \mid z^{(k)})\, p(z^{(k)})}{q_\phi(z^{(k)} \mid x)}\right), \qquad z^{(k)} \sim q_\phi(z \mid x),$$

implemented using log-space stabilization (*log-mean-exp*) to avoid numerical underflow.

Posterior diagnostics and collapse indicators (what we report). We report: (i) mean KL and distribution of per-sample KL, (ii) active units (dimensions where $\mathrm{Var}_x[\mathbb{E}(z \mid x)]$ exceeds a threshold), (iii) aggregate posterior statistics compared to the prior (mean/cov), (iv) latent traversals and PCA of inferred codes. These diagnostics are necessary because a low loss can coincide with poor latent utilization (collapse), especially in high-capacity decoders.

7.11 Conditional GAN for Data Augmentation (WGAN-GP)

Problem setting and non-overlapping splits. We study conditional generation on FashionMNIST under a limited-data regime. The training set is partitioned into two *disjoint* subsets:

$$\mathcal{D}_{\mathrm{GAN}} \cap \mathcal{D}_{\mathrm{CLS}} = \emptyset,$$

where $\mathcal{D}_{\mathrm{GAN}}$ trains the generator/critic and $\mathcal{D}_{\mathrm{CLS}}$ trains the downstream classifier. This prevents a key confounder: if GAN training images overlap classifier training images, the generator can memorize examples that artificially inflate augmentation gains.

Conditional generator used in this implementation. The generator G_ψ takes noise $z \sim \mathcal{N}(0, I)$ and a class label y and produces $x_{\mathrm{fake}} = G_\psi(z, y)$. In `deepgen/models/gan.py`, conditioning is implemented by *adding* a learned label embedding to the noise:

$$z' = z + e_G(y), \qquad x_{\mathrm{fake}} = G_\psi(z', y).$$

This is a simple and effective conditioning mechanism when embedding dimension equals z dimension. The generator maps z' through a fully-connected layer to a 7×7 feature map, upsamples to 14×14, applies self-attention, upsamples to 28×28, and outputs an image with tanh so that $x_{\mathrm{fake}} \in [-1, 1]$.

Projection discriminator (conditional critic). The critic $D_\omega(x, y)$ is a projection discriminator [6]. Let $h_\omega(x) \in \mathbb{R}^{d_f}$ be pooled features. The conditional score is

$$D_\omega(x, y) = s_\omega(h_\omega(x)) + \langle h_\omega(x), e_D(y) \rangle,$$

which enforces class-conditional discrimination without concatenating labels to image channels. The implementation uses global sum pooling (sum over spatial locations) to form $h_\omega(x)$ and then adds the projection term.

Spectral normalization and its role. To stabilize adversarial optimization, we apply spectral normalization (SN) to discriminator and generator layers [5]:

$$\bar{W} = \frac{W}{\sigma_{\max}(W)}.$$

SN controls operator norms and improves stability; empirically it reduces exploding critic gradients and can reduce sensitivity to learning-rate choices.

Attention mechanisms included in the architecture. We incorporate: (i) self-attention (SAGAN-style) to capture long-range dependencies [7], and (ii) channel attention (SE-style) to recalibrate channel responses [8]. Both are implemented as residual modules. Self-attention computes an $HW \times HW$ attention matrix to aggregate global context; SE uses global average pooling followed by a bottleneck MLP and a sigmoid gate to modulate channels.

WGAN-GP objective and regularization terms. We adopt WGAN-GP [4]. The critic estimates a Wasserstein difference:

$$\mathcal{W} \approx \mathbb{E}[D(x_{\text{real}}, y)] - \mathbb{E}[D(x_{\text{fake}}, y)].$$

The 1-Lipschitz constraint is enforced via gradient penalty on interpolated samples $\hat{x}$:

$$\hat{x} = \epsilon x_{\text{real}} + (1-\epsilon)x_{\text{fake}}, \qquad \epsilon \sim \mathcal{U}(0,1),$$

$$\mathcal{L}_{\text{GP}} = \lambda_{\text{GP}}\, \mathbb{E}_{\hat{x}} \left(\|\nabla_{\hat{x}} D(\hat{x}, y)\|_2 - 1 \right)^2 .$$

The implementation also includes: (i) a drift penalty to prevent critic output drift,

$$\mathcal{L}_{\text{drift}} = \epsilon_{\text{drift}}\, \mathbb{E}\left[D(x_{\text{real}}, y)^2 \right],$$

and (ii) embedding weight decay / regularization,

$$\mathcal{L}_{\text{emb}} = \lambda_{\text{emb}} \|e_D\|_2^2.$$

Thus, the critic loss minimized in training is:

$$\mathcal{L}_D = -\Big(\mathbb{E}[D(x_{\text{real}}, y)] - \mathbb{E}[D(x_{\text{fake}}, y)] \Big) + \mathcal{L}_{\text{GP}} + \mathcal{L}_{\text{drift}} + \mathcal{L}_{\text{emb}},$$

and the generator loss is:

$$\mathcal{L}_G = -\mathbb{E}[D(x_{\text{fake}}, y)].$$

Optimization protocol and logging. We use Adam with assignment hyperparameters and update the critic N_{critic} times per generator step. During critic updates, fake samples are detached to avoid unnecessary generator gradient computation. At each logging step we record: Wasserstein estimate, gradient penalty magnitude, drift term, embedding regularization, total losses, and fixed-noise sample grids for qualitative monitoring.

Augmentation evaluation (what constitutes valid evidence). To evaluate whether synthetic data improves generalization, we train a classifier in two regimes: (i) real-only using $\mathcal{D}_{\text{CLS}}$, and (ii) real+synthetic using $\mathcal{D}_{\text{CLS}}$ plus a controlled number of generated samples per class. Final evaluation is performed *only* on the untouched FashionMNIST test set (real samples). We report overall accuracy, class-wise accuracy, and confusion patterns. Any improvement must be interpreted jointly with generator diagnostics (fixed-noise grids and critic statistics) to distinguish genuine distributional expansion from mode-dropping artifacts.

8 Implementation

9 Implementation Details and Codebase Walkthrough

This section documents the code modules that implement the datasets, metrics, reproducibility utilities, and model architectures. The goal is to ensure that every reported experimental claim can be traced to a specific implementation component.

9.1 `deepgen/data.py`: Datasets, transforms, and loaders

Purpose. This module defines the preprocessing transforms and PyTorch `DataLoader` objects used throughout training and evaluation.

Transform definition: `mnist_transforms`. The function `mnist_transforms(scale_to_minus1_1)` composes:

- `ToTensor`: converts PIL images to float tensors in $[0, 1]$,
- optional `Normalize(mean=0.5,std=0.5)`: maps $[0, 1] \to [-1, 1]$ using $x \mapsto (x - 0.5)/0.5$.

This range convention is essential because the GAN generator outputs $\tanh(\cdot) \in [-1, 1]$, and the VAE encoder in this project is written assuming inputs in $[-1, 1]$. When the VAE reconstruction uses BCE against sigmoid outputs, targets are converted back to $[0, 1]$ as $(x + 1)/2$ inside the loss.

Loader construction: `get_mnist_loaders`. `get_mnist_loaders` downloads MNIST via `torchvision.datasets.MNIST` and returns two loaders (train/test). Key scientific/engineering choices:

- **Shuffling:** training data is shuffled to reduce gradient bias.
- **drop_last=True (train):** ensures consistent batch size; this is often important for stable adversarial training and for batchnorm behavior.
- **pin_memory:** enabled only when CUDA is available to speed host-to-device transfers.
- **train_subset:** allows controlled small-data experiments by restricting training indices to `range(train_subset)`; this must be logged to preserve interpretability of limited-regime results.

9.2 `deepgen/metrics.py`: Feature extraction and FID-like distance

Purpose. This module computes a Fréchet-distance-style metric in a learned feature space, analogous to FID, but using a project-specific feature extractor rather than Inception.

`extract_features`. Given a `feature_model` and a `DataLoader`, this function:

- switches to eval mode and disables gradients,
- forwards batches through the model with `return_features=True`,

- aggregates features into a NumPy array of shape (N, D).

This yields a deterministic feature matrix when seeds and threading are controlled.

Covariance and matrix square root. The helper `_covariance` computes empirical mean and covariance with Bessel correction. The function `_sqrtm_psd` computes the symmetric PSD square root using eigen-decomposition with eigenvalue clipping for numerical stability.

`fid_like`. Given real features and fake features, the metric is:

$$\|\mu_r - \mu_f\|_2^2 + \mathrm{Tr}\Big(\Sigma_r + \Sigma_f - 2(\Sigma_r^{1/2}\Sigma_f\Sigma_r^{1/2})^{1/2}\Big),$$

with diagonal jitter ϵI added to covariances. This is a standard Fréchet distance formula; in this project it should be interpreted as a *FID-like proxy* dependent on the chosen feature model (e.g., LeNet penultimate features), and must be reported with that caveat.

9.3 `deepgen/utils.py`: Reproducibility utilities and checkpointing

Determinism controls. `set_seed(seed)` sets Python, NumPy, and Torch seeds, and forces CuDNN determinism (at possible speed cost). This is required for scientifically defensible comparisons of GAN/VAE runs, since nondeterminism can otherwise dominate observed variance.

Device selection. `get_device("auto")` chooses CUDA when available, otherwise CPU, ensuring a single codepath with logged hardware dependence.

Atomic checkpoint saving. `_atomic_torch_save` writes checkpoints to a temporary file and then atomically renames it, preventing partial/corrupted checkpoints if the process crashes mid-write.

RNG state capture/restore. `capture_rng_state` and `restore_rng_state` store/restore Python/NumPy/Torch RNG states (and CUDA RNG if available). This enables exact continuation of training trajectories from checkpoints.

Checkpoint schema. `save_checkpoint` stores:

- model state dict,
- optimizer state dict (optional),
- extra metadata (optional),
- RNG state,
- torch version string.

This schema is sufficient for scientific reproducibility and audit trails.

AverageMeter. `AverageMeter` tracks running averages for losses/metrics; it is used to report stable epoch-level summaries rather than noisy per-step values.

9.4 `deepgen/models/blocks.py`: Reusable neural building blocks

SEBlock (channel attention). `SEBlock` implements squeeze-and-excitation gating: global average pooling creates a channel descriptor, then a bottleneck MLP and sigmoid gate produce channel-wise weights that modulate the input. This improves representational efficiency by emphasizing informative channels.

ResidualBlock. `ResidualBlock` provides two conv-bn layers plus an optional SE module and a skip path. When `downsample=True`, stride=2 is used and the skip path becomes a 1×1 strided conv. This supports deep encoders/decoders with stable gradients.

SelfAttention2d. `SelfAttention2d` computes attention across spatial positions (SAGAN-style). It builds query/key/value projections and uses a residual connection with a learnable scalar `gamma`, allowing the network to gradually introduce non-local interactions as training stabilizes.

9.5 `deepgen/models/classifier.py`: Feature extractor and classifier (LeNet5)

Purpose. `LeNet5` serves two roles: (i) a baseline classifier for MNIST-like data, and (ii) a feature extractor for `fid_like` when called with `return_features=True`.

Feature interface. The model exposes penultimate features (dimension 84) via `return_features`. This is the representation used for feature-space evaluation in `deepgen/metrics.py`. Any reported FID-like score must explicitly state that features are LeNet-based.

9.6 `deepgen/models/gan.py`: Conditional WGAN-GP with projection discriminator

Generator. The generator embeds labels and adds them to noise, projects to a 7×7 feature map, upsamples via residual-style blocks, applies self-attention, and outputs tanh-bounded images in $[-1, 1]$.

Discriminator (critic). The projection discriminator computes pooled features and adds an inner product with a label embedding. This is a standard conditional design that is parameter-efficient and empirically stable.

Gradient penalty. `gradient_penalty` constructs interpolates $\hat{x}$ and enforces $\|\nabla_{\hat{x}} D(\hat{x}, y)\|_2 \approx 1$. This is the core WGAN-GP regularizer for Lipschitz control.

9.7 `deepgen/models/vae.py`: VAE with VampPrior

Encoder/decoder. The encoder outputs $(\mu, \log \sigma^2)$. The decoder outputs sigmoid pixels in $(0, 1)$.

VampPrior pseudo-inputs. Pseudo-inputs are learned image-shaped parameters. Sampling selects a pseudo-input, encodes it to get $(\mu, \log \sigma^2)$, samples z, and decodes. The KL term is computed against the mixture prior using log-sum-exp, which is numerically stable and aligns with the VampPrior definition.

10 Evaluation and Metrics

10.1 Evaluation Objectives

Evaluation is designed to answer three scientific questions under controlled, reproducible conditions:

1. **VAE capacity and inference quality:** Does the modified VAE (e.g., flexible prior such as VampPrior, hierarchical latent design when enabled, and/or architectural capacity changes) improve latent utilization, sample quality, and likelihood-related metrics relative to a baseline VAE with a simple prior?
2. **Posterior collapse behavior:** How do KL dynamics, active units, and aggregate posterior statistics differ between baseline and proposed models, and do these differences indicate reduced collapse risk or more effective latent usage?
3. **Augmentation effectiveness:** Under a limited labeled-data regime, does conditional WGAN-GP augmentation improve downstream classifier generalization *on real test data* without contaminating the classifier training split or introducing systematic class confusion?

10.2 General Evaluation Protocol and Reproducibility Controls

Fixed splits and no leakage. For the augmentation experiment, all subset indices defining $\mathcal{D}_{\text{GAN}}$ and $\mathcal{D}_{\text{CLS}}$ are stored to disk and reused across runs. We enforce:

$$\mathcal{D}_{\text{GAN}} \cap \mathcal{D}_{\text{CLS}} = \emptyset,$$

and log the intersection size (must be zero) as an integrity check. For all experiments, the test set is never used for tuning.

Determinism and run metadata. Each run logs a `config.json` specifying (at minimum): random seed, dataset split paths, model hyperparameters, optimizer settings, number of steps/epochs, and evaluation settings (e.g., K_{IS} for IWAE). When feasible, we fix PyTorch and NumPy seeds and disable nondeterministic CuDNN behaviors. Reported results are either: (i) single-run results with a fixed seed plus full artifacts, or (ii) averages over multiple seeds with mean±std reporting.

Exported evaluation artifacts. All evaluation scripts export both raw per-sample metrics and summary tables:

- `../results/metrics/`: CSV/JSON logs (per-step and per-epoch), per-image/per-sample scores, and summary tables.

- `../results/figures/`: plots used in the report (loss curves, KL curves, histograms, sample grids).
- `../results/samples/`: generated samples and traversals (VAE samples, GAN fixed-noise grids).

This ensures traceability: each plot/table in the report corresponds to a stored file.

10.3 VAE Metrics and Diagnostics

We evaluate VAEs with complementary objectives: (i) predictive/likelihood-related evidence, (ii) representation and posterior diagnostics, and (iii) qualitative interpretability.

(V1) Loss decomposition and reporting conventions. We report the negative ELBO and its components:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{rec}} + \beta \cdot \mathcal{L}_{\text{KL}},$$

where $\mathcal{L}_{\text{rec}}$ is the reconstruction term (BCE for Bernoulli pixels) and $\mathcal{L}_{\text{KL}}$ is the KL divergence term. If a hierarchical latent model is used, we additionally report KL *per level* (e.g., $\text{KL}(z_1)$ and $\text{KL}(z_2)$) and the total KL. In practice we log:

- epoch/step, $\mathcal{L}_{\text{rec}}$, $\mathcal{L}_{\text{KL}}$ (and per-level KL if applicable), $\mathcal{L}_{\text{total}}$,
- batch statistics (mean, median) and dataset-level aggregates on validation/test.

Interpretation: improvements in $\mathcal{L}_{\text{total}}$ are only meaningful when both reconstruction and KL terms are analyzed, because a lower total loss can be achieved by collapsing KL to near-zero (posterior collapse) without learning useful latents.

(V2) Latent activity and active units. Latent utilization is measured using *active units*. A standard criterion is:

$$\text{AU}_j = \mathbb{I}\left(\text{Var}_x\left[\mathbb{E}_{q_\phi(z|x)}(z_j)\right] > \tau_{\text{AU}}\right),$$

and the number of active units is $\sum_{j=1}^{d} \text{AU}_j$. We also report per-dimension KL contributions:

$$\text{KL}_j \approx \mathbb{E}_x\left[\text{KL}\left(q_\phi(z_j \mid x) \,\|\, p(z_j)\right)\right],$$

(or the corresponding quantity for a learned mixture prior). Interpretation: a model with many inactive units (few AUs and near-zero KL_j for most dimensions) indicates collapse or over-regularization; a healthier latent representation exhibits non-trivial AUs and structured KL_j.

(V3) Posterior–prior match (aggregate statistics). We compute aggregate posterior statistics over a large sample of datapoints:

$$\mu_q = \mathbb{E}_x\left[\mathbb{E}_{q_\phi(z|x)}(z)\right], \qquad \Sigma_q = \text{Cov}_x\left(\mathbb{E}_{q_\phi(z|x)}(z)\right),$$

and compare to: (i) the standard normal prior ($\mu = 0$, $\Sigma = I$), or (ii) the learned prior (e.g., mixture implied by pseudo-inputs). We also visualize Σ_q in a PCA basis (eigen-spectrum and

leading components) to assess whether the posterior occupies a lower-dimensional subspace. Interpretation: strong mismatch can indicate either insufficient prior flexibility or an overly constrained posterior; a flexible prior (e.g., VampPrior) should reduce pathological mismatch while maintaining non-trivial latent structure.

(V4) Likelihood estimation via importance sampling (IWAE-style). Because ELBO is a lower bound, we estimate $\log p_\theta(x)$ using an importance-weighted estimator:

$$\widehat{\log p_\theta(x)} = \log \left(\frac{1}{K_{\mathrm{IS}}} \sum_{k=1}^{K_{\mathrm{IS}}} \frac{p_\theta(x \mid z^{(k)}) p(z^{(k)})}{q_\phi(z^{(k)} \mid x)} \right), \quad z^{(k)} \sim q_\phi(z \mid x).$$

For fairness, we use the same K_{IS} across models, and we report K_{IS} explicitly. We compute dataset-level averages and uncertainty estimates (e.g., mean±std across datapoints or seeds). Interpretation: higher (less negative) values indicate better likelihood, but must be interpreted alongside collapse diagnostics, because some architectures can optimize likelihood while learning poor latent semantics.

(V5) Qualitative samples and latent traversals. We export:

- unconditional samples: $z \sim p(z)$ then $x \sim p_\theta(x \mid z)$,
- latent traversals: vary one latent dimension while holding others fixed,
- PCA traversals: traverse along principal directions of inferred latent codes.

Interpretation: meaningful traversals suggest disentangled or at least interpretable factors; random/noisy traversals with low KL often correlate with collapse.

(V6) Pseudo-input visualization (VampPrior-specific). For VampPrior models, we visualize pseudo-inputs $\{u_k\}$ (after any range constraint such as tanh) and optionally their decoded reconstructions. We interpret whether pseudo-inputs cover multiple modes (diverse digit styles/clothing prototypes). Interpretation: diverse pseudo-inputs are consistent with a multimodal learned prior that can reduce over-regularization; highly similar pseudo-inputs can indicate poor utilization of the mixture capacity.

10.4 GAN Training Metrics (Conditional WGAN-GP)

We evaluate GAN training using both *objective-based* statistics and *artifact-based* qualitative monitoring.

(G1) Critic and generator losses (decomposed). We log:

- Wasserstein term: $\mathbb{E}[D(x_{\mathrm{real}}, y)] - \mathbb{E}[D(x_{\mathrm{fake}}, y)]$,
- critic total loss: Wasserstein term plus regularization,
- generator loss: $-\mathbb{E}[D(x_{\mathrm{fake}}, y)]$.

Interpretation: the Wasserstein estimate should not diverge; excessively large magnitudes, instability, or oscillations often indicate training pathologies or learning-rate mismatch.

(G2) Regularization terms and constraint satisfaction. We explicitly plot:

- gradient penalty magnitude and its moving average,
- drift penalty contribution,
- embedding regularization magnitude (if used).

Interpretation: gradient penalty should remain in a reasonable range; persistent extreme GP values may imply critic gradients are poorly conditioned or that Lipschitz constraints are not being satisfied.

(G3) Fixed-noise, fixed-label sample grids (checkpoint comparable). We export sample grids at fixed training steps (e.g., $\{0, 1000, 2000, 3000, 4000\}$ or assignment-specified checkpoints) using fixed noise vectors and a fixed label pattern covering all classes. This ensures that visual differences across grids reflect *model changes* rather than sampling noise. Interpretation: improvements should manifest as sharper class-appropriate structure and reduced mode collapse (diversity within each class).

(G4) Optional feature-space quality metric (FID-like proxy). If a consistent feature extractor is available (e.g., a fixed LeNet feature model), we compute a FID-like Fréchet distance between real and generated features. This is reported as a proxy quality measure and must be interpreted as *feature-model dependent.*

10.5 Augmentation Evaluation (Classifier-Based)

Augmentation is evaluated strictly on *real* test data to measure generalization, not memorization.

(A1) Baseline vs augmented training protocol. We train the same classifier architecture under two regimes:

- **Real-only baseline:** train on $\mathcal{D}_{\text{CLS}}$ only.
- **Augmented:** train on $\mathcal{D}_{\text{CLS}}$ plus N_{syn} synthetic samples per class generated by the trained conditional GAN.

All hyperparameters (optimizer, learning rate schedule, epochs, augmentations other than GAN) are held fixed to isolate the effect of GAN augmentation.

(A2) Performance endpoints and reporting. We report:

- overall test accuracy on the untouched FashionMNIST test set,
- class-wise accuracy (or per-class error),
- optional confusion matrix to diagnose systematic class confusion,
- confidence intervals or mean$\pm$std across seeds (recommended in limited-data settings).

Interpretation: a scientifically meaningful improvement is one that is statistically consistent across seeds and not driven by a single class. Confusion matrices are particularly informative for detecting augmentation-induced boundary shifts.

(A3) Interpreting augmentation gains with sample quality diagnostics. Any observed accuracy improvement is interpreted jointly with:

- conditional sample grids (class fidelity and diversity),
- critic diagnostics (stability and constraint satisfaction),
- class-wise performance (to detect whether some classes are harmed).

If augmentation improves accuracy but increases confusion in specific classes, we document this as a limitation and analyze whether the generator is producing ambiguous inter-class samples.

(A4) Failure modes and validity threats specific to augmentation. We explicitly document:

- **Mode dropping:** generator produces limited intra-class diversity; can yield limited or misleading augmentation benefit.
- **Label noise via conditional confusion:** generator outputs samples that visually resemble a different class than the conditioning label, potentially harming classifier training.
- **Distribution shift amplification:** synthetic samples may overemphasize artifacts (checkerboarding, texture bias) that shift classifier decision boundaries.

Mitigation is evidence-driven: for example, reducing N_{syn}, filtering low-quality samples, or improving conditional fidelity (architecture/regularization) is adopted only if it improves test performance and reduces class-wise harm.

11 Results and Analysis

11.1 Overview of Exported Artifacts and Traceability Contract

The evaluation pipeline exports *both* qualitative and quantitative artifacts to guarantee that every claim in this section can be traced to files on disk. We follow a strict "artifact contract": for each evaluated pair $(\mathcal{I}_T, \mathcal{I}_Q)$ we export (i) at least one overlay image, and (ii) one row in a per-sample metrics CSV. Dataset-level tables and report figures are derived *only* from these raw exports.

Directory structure. Artifacts are stored under:

- **Overlays:** `../results/overlays/` — warped template rendered into the photo frame.
- **Metrics:** `../results/metrics/` — per-sample and summary CSV/JSON logs.
- **Figures:** `../results/figures/` — plots used in this report (histograms, curves, ablations, runtime).

Missing CSV: `../results/metrics/summary.csv`
Export your metrics to this path or update the filename in the .tex file.

Table 1: Dataset-level summary metrics exported by the evaluation script.

- **Debug (optional):** `../results/matches/` — match visualizations, inlier masks, and RANSAC diagnostics.

Recommended file naming convention. To make provenance unambiguous, filenames should include a sample identifier and key configuration parameters (or a short run hash). For example:

- `overlays/{id}_{run_hash}_overlay.png`
- `matches/{id}_{run_hash}_matches.png`, `matches/{id}_{run_hash}_inliers.png`
- `metrics/per_sample.csv`, `metrics/summary.csv`, `metrics/config.json`

The `run_hash` can be computed from the config JSON (e.g., SHA1 of the serialized config) to ensure that artifacts are uniquely tied to a configuration.

Minimum required exported metrics per sample. At minimum, each evaluated pair should export one row with:

- identifiers: `sample_id`, `template_path`, `photo_path`
- matching: `n_kp_T`, `n_kp_Q`, `n_matches`, `n_inliers`, `inlier_ratio`
- geometry: `H_status` (ok/fail), `H_cond` (optional), `warp_area_ratio` (optional)
- accuracy (if GT exists): `MRE`, `MedRE`, (optional) `p90_err`, `p95_err`
- runtime: `t_pre`, `t_feat`, `t_match`, `t_ransac`, `t_warp`, `t_total` (all in ms)

If ground truth is unavailable, the evaluator must explicitly report that MRE/MedRE are not computed and the results are based on proxy metrics (e.g., inlier ratio and overlay inspection).

11.2 Quantitative Summary (Dataset-Level Aggregates)

A dataset-level summary is exported as `../results/metrics/summary.csv`. This table aggregates performance across all evaluated samples under a fixed configuration. To prevent misleading conclusions from heavy-tailed failures, we report robust statistics in addition to means.

Scientifically correct interpretation of the summary table. The summary table should contain (at minimum):

- **Mean reprojection error (MRE):** average geometric error in pixels. Sensitive to failures; can be dominated by outliers.
- **Median reprojection error (MedRE):** robust central tendency. If MedRE $\ll$ MRE, performance is bimodal or heavy-tailed.

- **Success rate at tolerance δ:** $\Pr(\text{MRE} < \delta)$ (or $\Pr(\text{MedRE} < \delta)$ if specified). Operationally interpretable.
- **Runtime median and P95:** median latency reflects typical performance; P95 captures tail latency critical for deployment.

Confidence intervals (recommended). If the dataset size is not large, or if you compare multiple configurations, report uncertainty:

- **Bootstrap CIs** for MRE/MedRE and success rate (e.g., 1,000 bootstrap resamples of per-sample metrics),
- and/or **multi-seed runs** (if any randomized steps exist) reporting mean±std across runs.

This prevents overclaiming based on a single dataset realization.

11.3 Qualitative Results: Overlay Inspection Protocol

Qualitative overlays are the primary sanity check that the estimated homography produces a visually consistent mapping.

What this figure demonstrates. Overlays provide evidence that the mapping is correct in the *image plane*:

- **Correct alignment:** template contours (edges, holes, fiducials) coincide with the photographed part geometry.
- **Local drift:** misalignment increasing toward image corners often indicates insufficient inlier support, poor conditioning of $\mathbf{H}$, or planarity violation.
- **Systematic offset:** consistent displacement across samples strongly suggests a deterministic preprocessing mismatch (scale convention, cropping origin, or inconsistent template normalization).

Selection policy for overlays (to avoid cherry-picking). To make qualitative reporting scientifically defensible, include at least:

- a random subset of samples (fixed random seed),
- and the worst-k samples by error (or by lowest inlier ratio if no GT exists),
- plus a small set of "typical" (median) cases.

This ensures that failure modes are visible and not hidden by selection.

11.4 Alignment Error Distribution and Success Curves

Distributional reporting is essential because feature-based pipelines typically show heavy-tailed failures.

What the error histogram demonstrates.

- **Tight unimodal near zero:** stable, repeatable estimation.
- **Heavy tail:** rare catastrophic misalignments; motivates reporting success rates and worst-case diagnostics.
- **Bimodality:** two regimes (success vs failure), often caused by lighting/clutter/occlusion thresholds or repetitive patterns.

Success-vs-tolerance curve (recommended). In addition to a single tolerance δ, export a success curve:

$$S(\delta) = \Pr(\text{MRE} < \delta), \quad \delta \in \{\delta_1, \ldots, \delta_m\}.$$

Plotting $S(\delta)$ reveals whether improvements are meaningful at strict tolerances (precision) or only at loose tolerances (coarse alignment). Export as: `../results/figures/success_curve.png` and `../results/metrics/success_curve.csv`.

11.5 Matching and Geometry Diagnostics (Beyond Error Alone)

Quantitative alignment error does not fully explain failures. We therefore report match/geometry diagnostics that correlate with success.

Inlier ratio and inlier count. Given RANSAC inliers $\mathcal{M}_{\text{in}}$ and matches $\mathcal{M}$, define:

$$r_{\text{in}} = \frac{|\mathcal{M}_{\text{in}}|}{|\mathcal{M}|}.$$

Low $|\mathcal{M}_{\text{in}}|$ or low r_{in} often predicts instability in $\mathbf{H}$.

Homography sanity checks (recommended and loggable). Even if RANSAC returns a matrix, it can be numerically or geometrically implausible. Log and optionally threshold:

- **Condition indicator:** a proxy such as $\kappa(\mathbf{H})$ or $\kappa(\mathbf{A})$ from DLT, if available.
- **Warped-corner validity:** warp the template corners and check that a reasonable fraction lie within the photo bounds.
- **Area ratio:** compare area of warped template quadrilateral to original; extreme ratios indicate degenerate solutions.

If a sample fails sanity checks, label `H_status=fail` and exclude from numeric error summaries (or report separately), while still saving overlays for diagnosis.

11.6 Runtime and Stage-Wise Profiling

Runtime must be reported with a clear measurement protocol to avoid misleading numbers.

Measurement protocol (must be stated). To produce credible runtime claims:

- report hardware and software versions (CPU/GPU, OpenCV build),
- use a warm-up phase (discard first n samples) to avoid cold-start bias,
- measure per-image latency with consistent resolution policy (M fixed),
- report **median and P95**, not only mean.

Interpreting bottlenecks.

- **Feature extraction dominated:** reduce M or ORB `nfeatures` / pyramid depth.
- **Matching dominated:** use stricter prefilters (distance percentile), cross-check, or reduce descriptors.
- **RANSAC dominated:** indicates low inlier ratio; improve match quality or ROI-gate to suppress background.

11.7 Ablation Study: Parameter Sensitivity and Stability Regions

A scientifically defensible system is not only accurate at one setting; it is stable over a neighborhood of parameters.

Ablation design (recommended). At minimum, sweep:

- ORB `nfeatures` (e.g., $\{500, 1000, 2000\}$),
- RANSAC threshold τ (pixels) (e.g., $\{2, 3, 5, 8\}$),
- match filtering (cross-check on/off; distance percentile).

For each setting, report: MedRE, success rate at δ, and runtime median/P95.

Interpreting stability. A robust operating regime is characterized by:

- a **plateau** in success rate (insensitive to small perturbations),
- low MedRE without extreme τ,
- acceptable runtime across the plateau.

The scientific goal is to select a configuration from the stability region, not an isolated peak that may overfit the dataset.

11.8 Failure-Case Analysis and Diagnostics

We explicitly document failures, since they define the validity domain and guide mitigations.

Failure taxonomy (report as counts). Classify failures into categories and report how many occur in the test set:

- **Texture ambiguity / repetition:** geometrically consistent but incorrect homography.

- **Low texture / blur:** too few reliable keypoints, low inlier count.
- **Glare / saturation:** keypoint instability due to suppressed gradients.
- **Occlusion / truncation:** insufficient overlap between template and photo.
- **Planarity violation:** systematic drift that cannot be eliminated by tuning.

Evidence-driven mitigations (activate only if metrics improve).

- **ROI gating:** restrict matching to the part region to suppress background outliers (expected to increase r_{in}).
- **Stricter filtering:** cross-check + distance percentile; optionally enforce spatial dispersion of inliers to reduce local clustering.
- **Illumination normalization:** CLAHE in grayscale if glare/low contrast is the dominant failure driver.
- **Template standardization:** enforce consistent scale/contrast/line thickness to increase descriptor repeatability.

A mitigation is accepted only if it increases success rate at the chosen tolerance and reduces tail failures (e.g., lower P95 error), not merely improves the mean.

11.9 Result Summary: What the Results Establish

Collectively, the exported artifacts support three scientifically testable claims:

1. **Geometric correctness (within the validity domain):** supported by low MedRE/MRE (when GT exists) and visually consistent overlays on a stratified set of samples.
2. **Robustness characteristics:** supported by error distributions (tail behavior), success-vs-tolerance curves, and explicit failure-case taxonomy with counts.
3. **Operational feasibility:** supported by median/P95 runtime and stage-wise bottleneck analysis under a fixed resolution policy.

What must be filled with real numbers. Replace placeholders using `../results/metrics/` exports:

- dataset size: [**REPLACE: N**]
- MedRE, MRE, P90/P95 error: [**REPLACE: MedRE**], [**REPLACE: MRE**], [**REPLACE: P95**]
- success at key tolerances: [**REPLACE: S(3px)**], [**REPLACE: S(5px)**], [**REPLACE: S(10px)**]
- runtime median/P95: [**REPLACE: median ms**], [**REPLACE: P95 ms**]
- failure counts by category: [**REPLACE: counts**]

All reported values must be directly traceable to the exported CSVs and figures.

12 Code Explanation

13 Codebase Walkthrough and Implementation Rationale

This section explains the implementation at the level of individual modules and functions. The codebase (`deepgen/`) supports three main workflows:

1. Data loading and normalization (MNIST loaders; consistent pixel ranges for GAN/VAE).
2. Model definitions: (i) VAE with VampPrior, (ii) conditional GAN with Projection Discriminator + SN + attention, and (iii) a classifier used for feature extraction.
3. Metrics, logging, determinism, and checkpointing for reproducible experiments.

Throughout the code, tensor shapes follow PyTorch conventions: images are (B, C, H, W) with $C = 1$ for MNIST-like grayscale inputs. When `scale_to_minus1_1=True`, inputs are mapped to $[-1, 1]$, which aligns naturally with GAN generators using $\tanh(\cdot)$.

13.1 `deepgen/data.py`: Data transforms and loaders

Purpose. This file constructs **consistent, reproducible** PyTorch dataloaders for MNIST and defines the input normalization convention used by downstream models.

`mnist_transforms(scale_to_minus1_1=True)`

- **Step 1:** `transforms.ToTensor()` converts a PIL image to a float tensor in $[0, 1]$ with shape $(1, 28, 28)$.
- **Step 2 (optional):** `Normalize(mean=(0.5,), std=(0.5,))` maps $[0, 1]$ to $[-1, 1]$ via

$$x' = \frac{x - 0.5}{0.5} = 2x - 1.$$

Rationale: GAN generators typically output $[-1, 1]$ through tanh, so training data in the same range avoids scale mismatch. For VAEs, the code later converts $[-1, 1]$ back to $[0, 1]$ when using a Bernoulli reconstruction likelihood.

`get_mnist_loaders(...)`

This function returns (`train_loader`, `test_loader`). Key details:

- **Dataset:** `datasets.MNIST(..., download=True, transform=tfm)` ensures reproducible access to raw MNIST with the exact preprocessing pipeline applied on-the-fly.
- **Optional subset:** if `train_subset` is provided, `Subset(train_ds, range(train_subset))` restricts training to the first N examples. This is useful for *limited-data* experiments (e.g., GAN augmentation studies).
- **Shuffling:** enabled for training (`shuffle=True`), disabled for test loader (`shuffle=False`).
- **pin_memory:** activated only when CUDA is available, improving host-to-device transfer throughput in GPU training.

- **drop_last:** `True` for training ensures a constant batch size, which simplifies batchnorm behavior and logging.

13.2 `deepgen/utils.py`: Reproducibility, checkpointing, bookkeeping, attention blocks

Purpose. This module provides (i) deterministic seeding, (ii) device selection, (iii) atomic checkpoint saving with RNG capture, and (iv) utility classes for stable logging. Your pasted content also includes attention implementations; conceptually, those are model blocks and can be relocated to `deepgen/models/blocks.py` for cohesion, but functionally they work as-is.

`set_seed(seed)`

Sets random seeds for:

- Python `random`
- NumPy RNG
- PyTorch CPU RNG
- PyTorch CUDA RNG (all devices)

and enforces deterministic CuDNN behavior:

`cudnn.deterministic=True`, `cudnn.benchmark=False`.

Rationale: This reduces run-to-run variation due to non-deterministic kernel selection or algorithm heuristics, improving scientific reproducibility. The trade-off is potential speed loss.

`get_device(device)`

- If `device` is `None` or `"auto"`, selects `"cuda"` when available, otherwise `"cpu"`.
- Otherwise constructs `torch.device(device)`.

Atomic checkpoint saving: `_atomic_torch_save`

Checkpoints are written to a temporary file and then renamed:

`torch.save(tmp)` $\rightarrow$ `os.replace(tmp, path)`.

Rationale: Prevents corrupted checkpoints if the process is interrupted mid-write.

`capture_rng_state()` and `restore_rng_state(state)`

The code stores and restores RNG states for Python, NumPy, PyTorch CPU, and (optionally) CUDA. This ensures that resuming from a checkpoint can reproduce the same stochastic sequence (important for exact reproducibility of sampling and training trajectories).

`save_checkpoint / load_checkpoint`

A checkpoint payload includes:

- model state dict
- optimizer state dict (optional)
- extra metadata (optional)
- RNG states (critical for reproducibility)
- PyTorch version

Scientific note: Recording `torch_version` helps interpret any numerical drift across environments.

`AverageMeter`

Tracks cumulative value and count, returning:

$$\texttt{avg} = \frac{\sum v_i}{N}.$$

Used for stable logging of losses/metrics over an epoch or steps.

Attention blocks in the pasted `utils.py`

`ChannelAttention`. Implements a Squeeze-and-Excitation-style mechanism:

1. Global average pooling: $(B, C, H, W) \to (B, C)$
2. Two FC layers with reduction r: $C \to \max(1, \lfloor C/r \rfloor) \to C$
3. Sigmoid gating to produce channel weights (B, C)
4. Broadcast-multiply back onto feature map: (B, C, H, W)

Effect: reweights channels based on global context, improving representational efficiency.

`SelfAttention2d`. SAGAN-style self-attention over spatial positions:

- Query: $q = \text{Conv}_{1\times1}(x) \in \mathbb{R}^{B\times C_q\times HW}$
- Key: $k = \text{Conv}_{1\times1}(x) \in \mathbb{R}^{B\times C_q\times HW}$
- Attention map: $\text{softmax}(q^\top k) \in \mathbb{R}^{B\times HW\times HW}$
- Value: $v = \text{Conv}_{1\times1}(x) \in \mathbb{R}^{B\times C\times HW}$
- Output: $o = v \cdot \text{Attn}^\top \to (B, C, H, W)$
- Residual: $y = x + \gamma o$ with learnable γ initialized at 0.

Rationale: $\gamma = 0$ initialization makes the network start close to the non-attention baseline and learn attention contributions gradually, improving optimization stability.

13.3 `deepgen/metrics.py`: Feature extraction and FID-like score

Purpose. This module computes a Fréchet-distance-like statistic between features of real and generated samples. It is useful for tracking generative quality without relying on pixel-level errors.

`extract_features(feature_model, loader, device, max_items)`

- Runs `feature_model.eval()` and disables gradients (`@torch.no_grad()`).
- For each batch x, calls `feature_model(x, return_features=True)` to obtain features $f \in \mathbb{R}^{B\times D}$.
- Concatenates across batches into a single $\mathbb{R}^{N\times D}$ array.

Important assumption: The model passed here (e.g., `LeNet5`) must implement `return_features=True` and return a stable embedding representation.

Covariance and PSD square root

`_covariance` computes μ and the unbiased covariance estimate:

$$\Sigma = \frac{1}{N-1}(X-\mu)^\top (X-\mu).$$

`_sqrtm_psd` computes $\Sigma^{1/2}$ by eigen-decomposition, clipping eigenvalues to ϵ to avoid numerical issues.

`fid_like(features_real, features_fake)`

Computes

$$\|\mu_r - \mu_f\|_2^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_f - 2(\Sigma_r^{1/2}\Sigma_f\Sigma_r^{1/2})^{1/2}\right).$$

Notes:

- This matches the standard Fréchet distance between Gaussians (the mathematical core of FID), but the *feature extractor* is your own model, not Inception.
- Small diagonal ϵI is added to covariances for stability.

13.4 `deepgen/models/blocks.py`: Core building blocks (Residual, SE, Self-Attention)

`SEBlock`. Implements channel gating via global mean pooling and a 2-layer MLP, then multiplies the gating vector back into the feature map. This is functionally equivalent to `ChannelAttention` in spirit (SE-style attention).

`ResidualBlock`. A ResNet-style block:

- Two 3×3 convs with BatchNorm.
- Optional downsampling via stride-2 in the first conv and skip path.

- Optional SE gating on the residual branch.
- Output: $\mathrm{ReLU}(h + \mathrm{skip}(x))$.

Rationale: Residual connections improve gradient flow and stabilize deeper encoders/decoders.

`SelfAttention2d`. Provides a second implementation of SAGAN-like attention (similar to the one in your pasted `utils.py`). For maintainability, it is best practice to keep a *single* canonical attention implementation and reuse it across GAN/VAE blocks.

13.5 `deepgen/models/classifier.py`: LeNet5 classifier as feature extractor

Purpose. `LeNet5` serves two roles:

1. A baseline classifier (logits output) for evaluating augmentation effects.
2. A **feature extractor** for `fid_like` via `return_features=True`.

Forward pass and shapes

Input $x \in \mathbb{R}^{B\times 1\times 28\times 28}$ (expected in $[-1, 1]$):

- `conv1(1→6, 5×5)`: $28 \to 24$
- avgpool 2×2: $24 \to 12$
- `conv2(6→16, 5×5)`: $12 \to 8$
- avgpool 2×2: $8 \to 4$
- flatten: $16 \cdot 4 \cdot 4 = 256$
- `fc1: 256→120`, `fc2: 120→84` (penultimate feature), `fc3: 84→10`

If `return_features=True`, the model returns the 84-D penultimate embedding.

13.6 `deepgen/models/gan.py`: Conditional GAN with SN, attention, and Projection Discriminator

High-level design. The GAN is conditional ($y \in \{0, \ldots, 9\}$), uses:

- Spectral normalization (SN) for Lipschitz control and stability.
- Self-attention to model long-range spatial dependencies.
- SE blocks for channel-wise reweighting.
- Projection discriminator for effective conditioning.
- WGAN-GP gradient penalty (implemented in `gradient_penalty`).

`GenBlock`

A residual upsampling block:

- Optional upsampling by factor 2 using nearest neighbor interpolation.

- Two SN conv layers with BatchNorm and ReLU.
- Optional SE gating on the residual branch.
- Skip path uses a 1×1 SN conv to match channels.

Rationale: Residual upsampling is a common stable pattern in modern GAN generators. Nearest-neighbor upsampling avoids checkerboard artifacts sometimes associated with transposed convolutions.

`Generator`

Key operations:

- **Conditional injection:** $\tilde{z} = z + \text{Emb}(y)$, where $\text{Emb}(y) \in \mathbb{R}^{z\_dim}$.
- **FC reshape:** $(B, z) \to (B, \text{base\_ch}, 7, 7)$.
- Upsample to 14×14 (`block1`), then apply self-attention, then upsample to 28×28 (`block2`).
- Output uses tanh to produce samples in $[-1, 1]$, matching the dataloader normalization.

`DiscBlock`

Residual downsampling block:

- SN conv with stride 2 for spatial downsampling (when enabled).
- Second SN conv.
- Skip connection with 1×1 SN conv and matching stride.
- LeakyReLU nonlinearity.

`ProjectionDiscriminator`

Implements projection conditioning:

$$D(x, y) = w^\top h(x) + b + \langle h(x), v(y) \rangle,$$

where $h(x) \in \mathbb{R}^{B \times C}$ is a pooled feature vector and $v(y)$ is a class embedding. In code:

- `features(x)` extracts conv features and applies global sum pooling:

$$h(x) = \sum_{i,j} \text{feat}(x)_{:,:,i,j}.$$

- `fc(h) + bias` gives the unconditional score.
- `proj = (embed(y) * h).sum(dim=1)` gives the inner product term.

Note: Global *sum* pooling (not mean pooling) increases the magnitude of features with spatial extent; this can be beneficial for some GAN setups but should be kept consistent across experiments.

`gradient_penalty(D, real_x, fake_x, y)`

Implements WGAN-GP penalty:

1. Interpolate $\hat{x} = \epsilon x_{\text{real}} + (1-\epsilon)x_{\text{fake}}$.
2. Compute $D(\hat{x}, y)$ and gradients $\nabla_{\hat{x}} D$.
3. Penalize deviation of gradient norm from 1:

$$\text{GP} = \mathbb{E}\left[(\|\nabla_{\hat{x}} D\|_2 - 1)^2\right].$$

Rationale: Enforces approximate 1-Lipschitz behavior needed for Wasserstein distance estimation, improving training stability.

13.7 `deepgen/models/vae.py`: VAE with VampPrior and ELBO

Model definition. The VAE defines a latent-variable model $p_\theta(x, z) = p_\theta(x|z)p(z)$. In this implementation, the prior $p(z)$ is replaced by a learned **VampPrior** mixture:

$$p(z) = \frac{1}{K}\sum_{k=1}^{K} q_\phi(z|u_k),$$

where u_k are learnable pseudo-inputs (images). This increases prior flexibility and can improve latent utilization.

`Encoder`

Input $x \in (B, 1, 28, 28)$. The encoder:

- Conv stem: $1 \to 32$.
- Residual downsample blocks: $28 \to 14 \to 7$, channels $32 \to 64 \to 128$.
- FC: $128 \cdot 7 \cdot 7 \to 256$.
- Outputs:

$$\mu_\phi(x) \in \mathbb{R}^{B\times z}, \quad \log \sigma^2_\phi(x) \in \mathbb{R}^{B\times z}.$$

`Decoder`

Maps $z \in \mathbb{R}^{B\times z}$ to $x_{\text{rec}} \in (B, 1, 28, 28)$:

- FC: $z \to 256 \to 128 \cdot 7 \cdot 7$.
- Residual block at 7×7.
- Two transposed conv upsampling steps: $7 \to 14 \to 28$.
- Output sigmoid:

$$x_{\text{rec}} = \sigma(\cdot) \in [0, 1].$$

Important consistency detail: even if training inputs are $[-1, 1]$, the decoder uses sigmoid and the reconstruction loss is Bernoulli/BCE on $[0, 1]$. The code therefore converts x to $[0, 1]$ before BCE.

`VAEVampPrior`: pseudo-inputs and prior evaluation

Pseudo-inputs. `self.pseudos` is a learnable tensor of shape $(K, 1, 28, 28)$. During sampling and prior computation, pseudo-inputs are passed through tanh to keep them in a bounded image-like range:

$$u_k = \tanh(\texttt{pseudos}_k) \approx [-1, 1].$$

Reparameterization. Given μ and $\log \sigma^2$, sample:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

`_log_normal(z, mu, logvar).` Computes $\log \mathcal{N}(z; \mu, \mathrm{diag}(\exp(\log \sigma^2)))$ per sample by summing over latent dimensions.

`log_pz_vampprior(z).` Computes:

$$\log p(z) = \log \left(\frac{1}{K} \sum_{k=1}^{K} q_\phi(z|u_k) \right) = \log \sum_{k=1}^{K} \exp(\log q_\phi(z|u_k)) - \log K,$$

implemented stably with `logsumexp`. This is the key difference from a standard normal prior.

ELBO loss: `elbo_loss(x, x_rec, mu, logvar, beta)`

The function computes:

- **Reconstruction term:** binary cross entropy over pixels. Since x may be in $[-1, 1]$, it is mapped to $[0, 1]$ via
$$x_{01} = \frac{x + 1}{2}.$$
- **KL-like term w.r.t. VampPrior:**
$$\mathrm{KL} = \log q_\phi(z|x) - \log p(z),$$
where $p(z)$ is VampPrior.
- **Final loss:**
$$\mathcal{L} = \mathbb{E}[\mathrm{Recon}] + \beta \cdot \mathbb{E}[\mathrm{KL}].$$

Implementation note: This function samples z internally using `reparameterize(mu, logvar)`. If `forward` already produced a sampled z, passing it through would reduce stochastic variance; however, the current approach remains correct as a Monte Carlo estimate.

Sampling: `sample(n, device)`

Sampling follows the VampPrior mixture logic:

1. Choose pseudo index $k \sim \text{Unif}\{1, \ldots, K\}$.
2. Compute $q_\phi(z|u_k)$ and sample z.
3. Decode z to obtain $x \in [0, 1]$.

This produces samples that reflect the learned prior mixture rather than a fixed standard normal.

13.8 Practical integration notes (what connects to what)

Data range conventions.

- GAN: generator outputs $[-1, 1]$ using tanh; discriminator expects the same.
- VAE: encoder ingests $[-1, 1]$ (if enabled), but decoder outputs $[0, 1]$ due to Bernoulli likelihood; BCE uses $[0, 1]$ targets.

Metric pipeline. `metrics.py` assumes a **feature model** with `return_features=True` (here `LeNet5`), enabling a stable embedding space for Gaussian statistics and the FID-like score.

Maintainability warning (duplicate attention blocks). Your pasted code contains `SelfAttention2d` in two places (once in the pasted `utils.py` block and again in `models/blocks.py`). For a clean scientific artifact, keep one canonical implementation (recommended: `models/blocks.py`) and import it everywhere to avoid silent divergence.

Computational cost note (VampPrior prior evaluation). `log_pz_vampprior` computes encoder outputs for all K pseudo-inputs, which can be expensive for large K. If runtime is an issue, a standard improvement is chunking pseudo-input evaluation or caching encoder outputs when the encoder parameters are not changing (not applicable during training but useful during evaluation).

14 Discussion

15 Discussion

15.1 Assumptions and Validity Domain

The proposed template-to-photograph alignment pipeline is a classical *feature-based registration* system whose core geometric model is a single planar projective transformation (homography) $\mathbf{H} \in \mathbb{R}^{3\times3}$ [**?**]. The modeling assumption is therefore:

> *All geometrically relevant points used for alignment lie on (or can be approximated by) a single plane in 3D space, as observed by a pinhole-like camera model.*

Under this assumption, a homography provides an exact mapping between two images of the same planar surface. In practice, "approximately planar" is acceptable when deviations from planarity produce reprojection errors below the operational tolerance δ (e.g., a few pixels at the working resolution). RANSAC-based estimation further assumes that a sufficient fraction of putative correspondences are inliers (i.e., consistent with a common $\mathbf{H}$) to enable robust recovery in the presence of outliers [**?**].

When the assumption is valid. The method is expected to perform well when:

- the imaged part face is flat (industrial plates, panels, PCB faces, printed labels), or the region of interest can be restricted to a near-planar patch;
- the camera viewpoint changes are moderate so that local appearance remains matchable by the chosen features (ORB by default);
- there is sufficient *repeatable local structure* (edges, corners, junctions, text/markings) to support stable keypoint detection and descriptor matching.

When the assumption is violated. A single homography becomes inadequate in common real-world cases:

- **Non-planar geometry / parallax:** curved surfaces, protrusions, and large depth variation induce parallax that cannot be modeled by one $\mathbf{H}$, producing systematic residuals even when correspondences are locally correct [**?**].
- **Strong occlusion / truncation:** insufficient overlap reduces the number of valid inliers, increasing estimator variance and failure probability.
- **Severe viewpoint changes:** appearance changes can break descriptor invariances, reducing match quality and inlier ratios.

Accordingly, the *validity domain* of the pipeline should be stated explicitly in the report: it is a robust planar registration method whose correctness is bounded by planarity, overlap, and feature repeatability.

15.2 Limitations and Failure Modes

The primary limitations arise from the interaction of (i) local-feature matching, (ii) robust estimation, and (iii) photometric/scene conditions.

(L1) Sensitivity to low texture, glare, and repetitive patterns. ORB is efficient and effective in many CPU-constrained scenarios, but its discriminative power depends on stable gradient structure and distinctive local neighborhoods [**?**]. In low-texture regions, keypoint density drops, and the correspondence set becomes too small for stable homography estimation. Specular highlights and glare can destroy repeatability by saturating intensities and altering local gradients. Repetitive patterns create *ambiguous* descriptors, increasing the probability of geometrically plausible but semantically incorrect homographies.

(L2) Degradation under large perspective changes and partial occlusions. Large viewpoint changes can (i) reduce overlap between template and photo, (ii) induce anisotropic scaling and foreshortening that harms descriptor matching, and (iii) lower the inlier ratio. Partial occlusions reduce the effective correspondence set and can cause RANSAC to select incorrect hypotheses due to insufficient inlier support [**?**].

(L3) Dependence on template quality and standardization. Because the template is one half of the matching problem, its rendering quality materially affects keypoint extraction and descriptor stability. Inconsistent line thickness, aliasing, poor contrast, or scale mismatches reduce matchability. Standardization (resolution normalization, contrast normalization, and consistent interpolation rules) is therefore not optional; it is a prerequisite for fair and stable evaluation.

(L4) Parameter sensitivity and latent "overfitting" via tuning. Even though the method is not learned end-to-end, its performance depends on parameters (resolution, ORB `nfeatures`, RANSAC τ, match filtering thresholds). Aggressive tuning on a small dataset can yield over-optimistic performance estimates. This is a classical threat in empirical computer vision: algorithmic knobs can overfit the evaluation distribution, especially when the dataset is narrow.

15.3 Mitigation Strategies (Evidence-Driven and Measurable)

Mitigations should be adopted only when they produce measurable gains (improved success rate at tolerance δ, reduced heavy-tail error behavior, improved inlier ratios) on a held-out evaluation set. The following interventions are technically justified and testable:

(M1) ROI gating to suppress background-induced outliers. If background clutter is a dominant failure driver, restricting correspondence search to a region of interest can increase inlier ratio and reduce catastrophic misalignments. ROI gating can be implemented via:

- manual bounding boxes (fast and controllable for small deployments),
- a lightweight detector/segmenter (for automation),
- template-derived masks (when applicable).

Empirically, ROI gating is justified if it reduces error tails and increases the fraction of successful alignments under the same δ.

(M2) Parameter selection via ablation stability regions. Rather than selecting a single "best" parameter setting, the scientifically defensible approach is to identify a *stable region* where performance changes minimally under small parameter perturbations. Ablations should report the joint effect on:

- accuracy metrics (MRE/MedRE and percentiles),
- robustness metrics (success rate, inlier ratio),

- efficiency metrics (median and P95 runtime).

A parameter choice is justified if it lies in a stable plateau of high success rate and acceptable runtime, rather than a sharp optimum.

(M3) Photometric normalization and template enhancement. When failures correlate with low contrast or glare, enable preprocessing steps whose effects can be measured:

- CLAHE in grayscale to enhance local contrast,
- mild denoising to stabilize gradients,
- template contrast normalization / edge enhancement to improve repeatable keypoints.

These should be treated as experimental factors (on/off) and included in ablations.

(M4) Stronger correspondence filtering. When ambiguous matches dominate, stricter filtering may improve geometric consistency:

- mutual nearest-neighbor cross-check,
- distance-based pruning (e.g., keep top-p percentile matches),
- sanity checks on spatial dispersion of matches (avoid degenerate clusters that yield unstable $\mathbf{H}$).

Filtering is justified only if it increases inlier ratio and success rate without collapsing match count below the minimum needed for stable estimation.

15.4 Threats to Validity and How the Protocol Addresses Them

We distinguish threats to *measurement validity* (whether metrics reflect reality) and *external validity* (generalization to unseen conditions).

(T1) Annotation noise and metric reliability. Reprojection error depends on ground-truth correspondences. Human annotation introduces uncertainty due to pixel-level ambiguity (corner definition, blur, and scale). This threat is mitigated by:

- using $K \geq 4$ non-collinear points, preferably more, to stabilize summary statistics;
- reporting robust aggregates (median, percentiles) alongside the mean;
- pairing numeric metrics with overlay inspection for face validity.

(T2) Dataset bias and incomplete coverage of real-world conditions. If the evaluation dataset under-represents critical conditions (e.g., harsh glare, heavy clutter, extreme viewpoint), measured robustness will be overestimated. Mitigation requires:

- deliberate acquisition spanning viewpoint, illumination, clutter, and occlusion regimes;
- reporting error distributions (not only averages) to expose rare failures;
- explicit inclusion of worst-case and representative failure figures.

(T3) Over-tuning and information leakage. Repeated parameter adjustments using the evaluation set can inflate reported performance. The recommended protocol is:

- **tuning split:** used for selecting parameters (resolution, `nfeatures`, τ, filtering rules);
- **held-out test split:** used once for final reporting;
- saving split indices and configuration logs to ensure repeatability and auditability.

(T4) Runtime measurement variability. Runtime can vary due to OS scheduling, CPU frequency scaling, and multithreading. This is mitigated by:

- repeated timing runs and reporting median/P95 rather than single measurements,
- fixing OpenCV thread count and logging hardware/software versions,
- evaluating at a fixed image resolution and fixed configuration.

15.5 Interpretation Guidelines for Readers

To avoid overclaiming, results should be interpreted through the lens of the model assumptions:

- If the method achieves low MedRE but higher mean error, the system is generally accurate but has tail failures; operational deployment must then rely on success thresholds and failure detection (e.g., low inlier ratio triggers fallback).
- If both mean and median errors are high, the likely cause is assumption violation (non-planarity) or systematic preprocessing/template mismatch; mitigation should focus on modeling changes or data preparation rather than parameter tweaks.
- If runtime meets median constraints but P95 is high, the system may be acceptable only with additional safeguards (e.g., cap iterations, early termination, ROI gating) to control pathological cases.

Overall, the discussion establishes a transparent boundary of validity, enumerates measurable limitations, and specifies mitigation strategies that can be empirically validated within the report's evaluation framework.

16 Reproducibility

16.1 Environment Specification

To guarantee that the experiments are fully reproducible, the following environment details should be recorded and included with the results:

- **Operating System (OS):** The version of the OS used during the experiment (e.g., Ubuntu 20.04, macOS 10.15).
- **Hardware:** The CPU model, RAM size, and GPU (if used) details.
- **Python version:** The version of Python used for the implementation.

- **OpenCV version and build:** The specific version of OpenCV (including CPU/GPU configuration) used for feature detection and matching.
- **Dependency lock file:** A lock file (e.g., `requirements.txt` or `environment.yml`) to capture all necessary Python dependencies.

16.2 Determinism Controls

Even well-established computer vision pipelines can show small nondeterminism due to threading and internal heuristics. To control for this, the following actions should be taken:

- Set fixed random seeds in Python, NumPy, and PyTorch to ensure that the random elements of the pipeline do not introduce variance.
- Set OpenCV's thread count to a fixed value (e.g., `cv2.setNumThreads(1)`) to ensure deterministic behavior.
- Log all parameter values and configurations to a `config.json` file, which can be referenced for reproducibility.

16.3 Reproducible Command Lines

To allow others to reproduce the results exactly, the following command lines should be provided. These commands include paths to data, output directories, and configuration files:

The complete reproducibility includes the exact dataset paths, configuration files, and thresholds used during evaluation.

16.4 File-Based Output Contract

For reproducibility and ease of use, each experimental run should generate specific output files:

- `config.json:` Contains all configuration parameters used in the run, including preprocessing settings, model hyperparameters, and evaluation thresholds.
- `curves/`: Contains CSV files of the training and validation loss curves and other relevant metrics (e.g., MRE, success rate).
- `samples/`: Contains generated samples (e.g., overlay images, augmented data).
- `checkpoints/`: Contains model weights and optimizer state files.
- `metadata.json:` Contains device details (e.g., GPU model), library versions, and dataset split hashes to ensure full reproducibility.

These files provide a complete record of the experimental process, ensuring that the results are fully reproducible.

17 Conclusion

18 Conclusion

This work implemented and evaluated a feature-based template-to-photograph alignment and overlay pipeline grounded in established multi-view geometry and robust model fitting. The method computes a planar projective mapping (homography) from template coordinates to photograph coordinates using local feature correspondences and RANSAC-based outlier rejection [**?** **?**]. The resulting mapping is used to warp the template into the camera view and produce geometrically consistent overlays for inspection, alongside quantitative metrics that support scientific assessment and reproducibility.

18.1 Summary of Method and Experimental Evidence

The system was designed as a modular pipeline with explicit exported artifacts (overlays, per-image metrics, distributions, and ablations), ensuring that all reported claims are traceable to stored outputs. Quantitatively, alignment quality is summarized using reprojection error statistics (mean and median) and operational success rates under application-specific tolerances, while runtime profiling provides feasibility evidence through median and tail latency (e.g., 95th percentile). Qualitatively, overlay grids provide a complementary validity check because small geometric deviations can be perceptually salient even when aggregate metrics appear acceptable.

Across the evaluation protocol, the results demonstrate that the pipeline can produce stable alignments under the validity domain of its modeling assumptions, particularly when (i) the imaged part face is approximately planar, (ii) sufficient repeatable local structure exists for feature matching, and (iii) preprocessing preserves discriminative gradients. The combination of distributional metrics (error histograms), ablations (parameter sensitivity), and explicit failure-case diagnostics enables a defensible characterization of robustness rather than relying on isolated examples.

18.2 Key Findings and Practical Implications

The experimental analysis supports the following conclusions:

Geometric correctness under the planar assumption. When the target surface is planar (or sufficiently close within the application tolerance), a single homography provides an accurate and compact model of the cross-view mapping [**?**]. In these cases, the method produces overlays with strong edge/contour coincidence and low reprojection error, indicating geometric consistency.

Robustness is bounded by feature repeatability and scene conditions. Performance degrades in conditions that reduce keypoint repeatability or increase correspondence ambiguity, including strong glare/specularities, low-texture regions, motion blur, and repetitive patterns. These effects manifest in the error distribution as heavy tails and in failure-case overlays as

gross misalignment. Importantly, such failures are not merely “noise”; they define operational boundary conditions of feature-based registration in unconstrained scenes.

Preprocessing and template quality are first-order factors. Template contrast, line thickness, and consistent scale normalization materially affect feature density and descriptor stability. Similarly, preprocessing policies (resize resolution, optional local contrast enhancement such as CLAHE) mediate a fundamental trade-off between runtime and matchability. The ablation study is therefore not auxiliary; it is required to justify configuration choices empirically and to identify a stable operating region rather than a fragile single optimum.

Mitigation strategies must be evidence-driven. ROI gating (e.g., restricting correspondence search to a detected/segmented part region) and stricter match filtering can reduce catastrophic tail failures by suppressing background-induced mismatches. However, such additions should be adopted only when they measurably improve success rates and reduce error tails on a representative evaluation set, consistent with scientific methodology that distinguishes hypothesized improvements from demonstrated gains.

18.3 Limitations and Boundary Conditions

Despite strong performance within its intended domain, the method has intrinsic limitations:

- **Non-planarity:** When the observed surface contains significant depth variation (curved parts, strong relief, or perspective-induced parallax across depth layers), a single homography cannot model the mapping exactly, and residual errors persist even under correct correspondences [**?**].
- **Ambiguous correspondences:** Repetitive textures can yield geometrically self-consistent but semantically incorrect homographies, particularly when incorrect matches concentrate in a locally coherent region.
- **Illumination pathologies:** Specular glare and low contrast reduce keypoint repeatability, lowering inlier counts and destabilizing RANSAC estimation [**?**].
- **Evaluation dependence on ground truth quality:** Reprojection metrics rely on accurate point annotations; label noise can inflate measured errors. This is mitigated by reporting robust statistics (median, percentiles) and by pairing quantitative results with overlay inspection.

These limitations define where the method should be used as-is (planar inspection tasks with sufficient structure) versus where extensions are required (non-planar targets, severe lighting artifacts, heavy clutter).

18.4 Future Directions

Several technically well-motivated directions can extend the method’s validity domain and robustness:

- **Non-planar alignment models:** For parts with depth variation, replace the single-homography assumption with models that represent 3D structure, such as multi-homography (piecewise planar) registration, pose estimation with known CAD geometry (PnP), or dense alignment methods that explicitly model parallax. The choice should be driven by the application's tolerance and available priors (e.g., known camera intrinsics, CAD correspondences).
- **ROI-aware matching in clutter:** Integrate a coarse detector or segmentation gate to restrict correspondence search to the part region. This reduces background-induced false matches and can significantly improve tail robustness, particularly when workbench clutter dominates failure modes.
- **Illumination-robust preprocessing:** Systematically evaluate photometric normalization strategies (e.g., CLAHE, glare suppression, gradient-domain representations) under controlled lighting variations, reporting effects on both accuracy and inlier ratios.
- **Deployment optimization:** If runtime constraints are strict, adopt configuration choices supported by ablation stability regions (e.g., reduced resolution and bounded `nfeatures`), precompute template descriptors, and consider approximate nearest-neighbor matching where it does not degrade success rate. Report median and tail latency to ensure rare pathological cases do not dominate operational behavior.

Overall, the presented pipeline provides a scientifically grounded baseline for template-to-photo alignment, with a complete evaluation methodology that characterizes both typical performance and boundary conditions. The exported artifacts and reproducible structure enable direct extension to more challenging scenarios while preserving traceability and experimental rigor.

Acknowledgements

The author would like to thank their advisor (Dr. [Advisor's Name]) and the research team at [Institution/Company Name] for their invaluable guidance and support throughout this project. Their insights were crucial in refining the methodology and experimental design. Special thanks are also due to [Collaborators' Names] for their feedback on the early versions of the system and their contributions to the codebase.

The author also acknowledges the open-source literature on feature-based image registration and robust estimation that served as the scientific foundation for this work, particularly the seminal works on multi-view geometry [**?**], robust estimation [**?**], and feature matching [**?**]. Their pioneering contributions to these fields have made the implementation of this system possible. Additionally, the availability of datasets such as MNIST and FashionMNIST has provided a standard benchmark for evaluating generative models, and the community's contributions in these areas are greatly appreciated.

The author would like to extend gratitude to [Funding Agency] for the financial support, which made the research feasible. Lastly, thanks to family and friends for their continued support and encouragement during the course of this project.

A Appendix

B Appendix

B.1 Recommended Metrics File Formats

For scientific reproducibility, every evaluation run should export machine-readable logs with (i) clear semantics, (ii) stable column names, and (iii) explicit units. We recommend producing both a **per-image metrics file** and a **dataset-level summary file**. This separation enables (a) aggregate reporting in the main paper and (b) post-hoc analyses such as error distributions, tail risk, and failure clustering.

A. Per-image metrics: `../results/metrics/per_image.csv`

Each row corresponds to a single evaluated photo-template pair. This file supports scientific analyses beyond averages (e.g., heavy-tail detection, subgroup analysis by lighting/viewpoint, and failure-case extraction).

Required columns (minimum scientific set).

- `image_id`: filename or unique identifier of the query photo.
- `template_id`: filename or unique identifier of the template used.
- `mre_px`: mean reprojection error (pixels) computed on ground-truth correspondences.
- `medre_px`: median reprojection error (pixels).
- `success_delta`: binary success indicator at the report tolerance δ (e.g., 5 px), i.e., $\mathbb{I}(\text{mre\_px} < \delta)$.
- `n_matches`: number of raw descriptor matches before RANSAC.
- `n_inliers`: number of RANSAC inliers.
- `inlier_ratio`: `n_inliers / max(1, n_matches)`.
- `runtime_ms`: total end-to-end runtime for this sample (milliseconds).

Recommended columns (diagnostics).

- `tau_px`: RANSAC reprojection threshold τ used for this run (pixels).
- `H_00`...`H_22`: flattened homography matrix entries (optional; for debugging/regression testing).
- `roi_enabled`: whether ROI gating was used (0/1).
- `clahe_enabled`: whether CLAHE was used (0/1).
- `failure_reason`: string label if alignment fails (e.g., `too_few_inliers`, `degenerate_H`, `missing_gt`).
- `overlay_path`: relative path to the saved overlay image for this sample.

Example (`per_image.csv`).

```
image_id,template_id,mre_px,medre_px,success_delta,n_matches,n_inliers,inlier_ratio,runtime_
photo_001.jpg,template_A.png,1.82,1.55,1,412,176,0.427,118.4,../results/overlays/photo_001_
photo_002.jpg,template_A.png,13.20,12.70,0,95,12,0.126,97.1,../results/overlays/photo_002__
```

B. Dataset summary: `../results/metrics/summary.csv`

The dataset-level summary is what the report tables should ingest (via `\SafeCSVTable`). It must include both central tendency and tail statistics to support deployment-relevant conclusions.

Required summary fields (minimum scientific set).

- `N`: number of evaluated image pairs.
- `MRE_mean_px`, `MRE_median_px`: mean and median of per-image MRE.
- `MedRE_median_px`: median of per-image MedRE (optional but recommended).
- `success_rate_{delta}px`: success rate at threshold δ (e.g., `success_rate_5px`).
- `runtime_median_ms`: median runtime per image.
- `runtime_p95_ms`: 95th percentile runtime (tail latency).

Recommended summary fields (robustness and tail risk).

- `MRE_p90_px`, `MRE_p95_px`: tail error percentiles.
- `inlier_ratio_mean`, `inlier_ratio_median`: correspondence consistency indicators.
- `fail_rate`: fraction of samples with failure label $\neq$ empty.

Example (`summary.csv`).

```
metric,value
N,200
MRE_mean_px,2.35
MRE_median_px,1.78
MRE_p95_px,8.92
success_rate_5px,0.91
inlier_ratio_mean,0.39
runtime_median_ms,112.6
runtime_p95_ms,176.4
fail_rate,0.04
```

C. Run manifest: `../results/metrics/run_manifest.json` (recommended)

To ensure full traceability across multiple experiments (ablations, parameter sweeps), each run should export a manifest linking `config.json`, metrics files, and figure artifacts.

Example (`run_manifest.json`).

```
{
  "run_id": "2026-01-26T09-12-31",
  "git_commit": "a1b2c3d",
  "config_path": "../results/metrics/config.json",
  "per_image_path": "../results/metrics/per_image.csv",
  "summary_path": "../results/metrics/summary.csv",
  "figures_dir": "../results/figures/",
  "overlays_dir": "../results/overlays/"
}
```

B.2 Annotation Schema (Point Correspondences)

Quantitative geometric evaluation requires ground-truth correspondences between template and photograph. We recommend a single JSON object per image pair stored under `../data/annotations/`. The schema below is designed to be minimal, explicit, and extensible.

A. Minimal schema

```
{
  "image_id": "photo_001.jpg",
  "template_id": "template_A.png",
  "points_template": [[x1,y1],[x2,y2],[x3,y3],[x4,y4]],
  "points_photo":    [[u1,v1],[u2,v2],[u3,v3],[u4,v4]]
}
```

Scientific requirements.

- At least four point pairs are required to evaluate a homography-driven mapping meaningfully; points must be non-collinear to avoid degeneracy.
- For reliable evaluation, points should be chosen on **repeatable landmarks** (corners, holes, endpoints of edges) rather than texture-only regions.
- If the image is resized for matching, either (i) annotate after resizing, or (ii) store the original resolution and the exact resize transform so evaluation occurs in a consistent coordinate system.

B. Recommended extended schema (for rigorous reporting)

For robust analysis and correct coordinate handling across resizing/cropping, we recommend adding explicit metadata:

```
{
  "image_id": "photo_001.jpg",
  "template_id": "template_A.png",
```

```
  "image_size_original": [W0, H0],
  "template_size_original": [WT0, HT0],
  "image_size_used": [W, H],
  "template_size_used": [WT, HT],
  "preprocess": {
    "resize_policy": "max_dim",
    "max_dim": 1024,
    "clahe": false,
    "roi": null
  },
  "points_template": [[x1,y1],[x2,y2],[x3,y3],[x4,y4],[x5,y5]],
  "points_photo":    [[u1,v1],[u2,v2],[u3,v3],[u4,v4],[u5,v5]],
  "annotator": "rater_01",
  "timestamp": "2026-01-26T09:00:00"
}
```

Why these fields matter.

- `image_size_used` and `template_size_used` prevent silent evaluation bugs caused by inconsistent resize conventions.
- `annotator` enables inter-rater analysis if multiple annotators are used.
- `preprocess` metadata enables causal debugging (e.g., whether CLAHE changes landmark visibility).

B.3 Parameter Logging (`config.json`)

To reproduce results exactly, every run must export a single configuration file (recommended path: `../results/metrics/config.json`). This file should capture all parameters that can affect outputs, including preprocessing, feature extraction, matching, RANSAC settings, and runtime controls.

A. Required configuration fields

- **Data and I/O**
  - `templates_dir`, `photos_dir`, `annotations_dir`
  - `results_dir`
- **Preprocessing**
  - `max_dim` (maximum image dimension used for matching)
  - `grayscale` (should be true for ORB in this pipeline)
  - `clahe_enabled`, and CLAHE parameters if enabled (clip limit, tile grid size)
  - `roi_enabled` and ROI source (manual box, detector name, mask path), if used

- **ORB parameters**
  - `orb_nfeatures`
  - `orb_scaleFactor`, `orb_nlevels`
  - `orb_edgeThreshold`, `orb_patchSize` (if modified)
- **Matching parameters**
  - `matcher` (e.g., BF-Hamming)
  - `cross_check` (0/1)
  - `distance_filter` (e.g., percentile threshold)
  - `distance_percentile` (if applicable)
- **RANSAC / homography**
  - `ransac_tau_px` (reprojection threshold τ)
  - `ransac_maxIters`
  - `ransac_confidence`
  - `min_inliers` (for declaring success/failure)
- **Overlay**
  - `alpha` (blending parameter)
  - `overlay_color` (if fixed color is used; otherwise omit)
- **Evaluation**
  - `delta_px` (success threshold δ)
  - `metrics` (list of computed metrics; e.g., `["mre","medre","inlier_ratio","runtime"]`)
  - `iou_enabled` (0/1) and mask paths if enabled
- **Runtime and determinism**
  - `seed`
  - `opencv_num_threads`

B. Example `config.json`

```
{
  "seed": 123,
  "opencv_num_threads": 1,
  "paths": {
    "templates_dir": "../data/templates",
    "photos_dir": "../data/photos",
    "annotations_dir": "../data/annotations",
    "results_dir": "../results"
```

```
  },
  "preprocessing": {
    "max_dim": 1024,
    "grayscale": true,
    "clahe_enabled": false,
    "roi_enabled": false
  },
  "orb": {
    "nfeatures": 2000,
    "scaleFactor": 1.2,
    "nlevels": 8
  },
  "matching": {
    "matcher": "bf_hamming",
    "cross_check": true,
    "distance_filter": "percentile",
    "distance_percentile": 90
  },
  "homography": {
    "ransac_tau_px": 3.0,
    "ransac_maxIters": 5000,
    "ransac_confidence": 0.995,
    "min_inliers": 20
  },
  "overlay": {
    "alpha": 0.45
  },
  "evaluation": {
    "delta_px": 5.0,
    "iou_enabled": false
  }
}
```

C. Scientific reproducibility checklist (what must be present)

To claim reproducibility, each reported experiment must have:

- `config.json` with all parameters used,
- `per_image.csv` enabling re-computation of summary stats,
- `summary.csv` included in the report table,
- overlays and figures referenced in the report saved under the declared paths,
- a run identifier (timestamp or hash) linking config, metrics, and artifacts.

References

[1] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2014.

[2] Danilo J. Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic backpropagation and approximate inference in deep generative models. *arXiv preprint arXiv:1401.4082*, 2014.

[3] Yuri Burda, Roger Grosse, and Ruslan Salakhutdinov. Importance weighted autoencoders. *arXiv preprint arXiv:1509.00519*, 2016.

[4] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron Courville. Improved training of wasserstein gans. *arXiv preprint arXiv:1704.00028*, 2017.

[5] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. *arXiv preprint arXiv:1802.05957*, 2018.

[6] Takeru Miyato and Masanori Koyama. cgans with projection discriminator. In *International Conference on Learning Representations (ICLR)*, 2018.

[7] Han Zhang, Ian Goodfellow, Dimitris Metaxas, and Augustus Odena. Self-attention generative adversarial networks. *arXiv preprint arXiv:1805.08318*, 2019.

[8] Jie Hu, Li Shen, and Gang Sun. Squeeze-and-excitation networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[9] Jakub M. Tomczak and Max Welling. Vae with a vampprior. *arXiv preprint arXiv:1705.07120*, 2018.